

BanglaSpeechFM: Towards a Foundation Model for Bangla Speech

by

Tamanna Afroz
23366033

A thesis submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
M.Sc. in Computer Science & Engineering

Department of Computer Science and Engineering
BRAC University
August 2026

© 2026. BRAC University
All rights reserved.

Declaration

It is hereby declared that

1. The thesis submitted is my/our own original work while completing degree at BRAC University.
2. The thesis does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
3. The thesis does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.
4. We have acknowledged all main sources of help.

Student's Full Name & Signature:

Tamanna Afroz
23366033

Approval

The thesis/project titled “Unlearning to Protect: A Distilled Reinforcement Learning Framework with Privacy-Preserving Feature Unlearning and XAI for IoT Security” submitted by

1. Tamanna Afroz (23366033)

Of Fall, 2025 has been accepted as satisfactory in partial fulfillment of the requirement for the degree of M.Sc. in Computer Science on November 91th, 2025.

Examining Committee:

Examiner:
(External)

Name
Professor
Department of Computer
Science and Engineering
BRAC University

Examiner:
(Internal)

Name
Professor
Department of Computer
Science and Engineering
BRAC University

Supervisor:
(Member)

Md. Golam Rabiul Alam, PhD
Professor
Department of Computer
Science and Engineering
BRAC University

Program Coordinator:
(Member)

Md Sadek Ferdous, PhD
Professor
Department of Computer
Science and Engineering
BRAC University

Head of Department:
(Chair)

Sadia Hamid Kazi, PhD
Chairperson and Associate
Professor
Department of Computer
Science and Engineering
BRAC University

Abstract

Botnets pose a significant cybersecurity threat, enabling attacks such as DDoS, data theft, and service disruptions on IoT devices. These devices often lack built-in botnet traffic filtering, leaving them highly exposed. Existing AI-based solutions improve detection capabilities but have limitations: (i) they are too heavy for IoT deployment, and (ii) they lack unlearning capabilities to forget sensitive or outdated features without retraining. To address these challenges, we propose DiRLU, a lightweight, reinforcement learning driven framework, while ensuring privacy by selectively unlearning sensitive or outdated features without requiring retraining. The framework leverages knowledge distillation to transfer knowledge from a teacher model into a lightweight student model, with both models trained using A2C. A post-hoc unlearning mechanism modifies weights to remove targeted features, while restored features show negligible performance loss, confirming reversibility. Unlike many benchmark models that used only 5% of the BoT-IoT dataset, this research leverages 25%, allowing us to develop a strong teacher model. Both the teacher and student models were trained using the A2C reinforcement learning algorithm, achieving impressive results, with the student model achieving 99.60% accuracy and a 99.80% F1 score. To enhance transparency, we integrated Explainable AI (XAI), particularly LIME, which helps interpret the model's decisions and identify the key features influencing its predictions. DiRLU combines efficiency with privacy, aligning with GDPR standards (right to be forgotten) to provide practical IoT security. By combining knowledge distillation, feature unlearning and XAI, this research not only strengthens botnet detection but also sets new standards for security, interpretability, and data privacy in cybersecurity.

Keywords: Botnet, Reinforcement Learning, Feature Unlearning, Knowledge Distillation, Explainable AI, A2C, Security & Privacy

Acknowledgement

Firstly, all praise to the Great Allah, by whose grace our thesis has been completed. We express our heartfelt gratitude to our supervisor Professor Golam Robiul Alam Sir for his kind support, valuable guidance, feedback and constant encouragement throughout this work. Our appreciation goes to the Department of Computer Science and Engineering of BRAC University for providing the necessary facilities and support.

Table of Contents

Declaration	i
Approval	ii
Abstract	iv
Acknowledgment	v
Table of Contents	vi
List of Figures	viii
List of Tables	x
Nomenclature	x
1 Introduction	1
1.1 Problem Statement	2
1.2 Thesis Objective	2
1.3 Thesis Outline	3
2 Related Work	4
3 Methodology	9
3.1 Data Description	9
3.2 Data Pre-Processing	10
3.2.1 Feature Pruning	10
3.2.2 Label Encoding	10
3.3 Data Split	11
3.4 Data Augmentation	12
3.5 Data Normalization	13
3.6 Reinforcement Learning	14

3.7	Knowledge Distillation	15
3.7.1	Teacher Model	16
3.7.2	Student Model	17
3.8	Un-learning	18
3.8.1	Teacher Unlearn	19
3.8.2	Student Unlearn	20
3.9	Explainable Artificial Intelligence (XAI)	20
3.10	Model Evaluation	21
3.11	Proposed Algorithm	21
4	Result and Analysis	24
4.1	Reinforcement Learning Results	24
4.2	Feature Unlearning Results	27
4.3	XAI: Teacher, Student and Post-Unlearning	30
4.4	Analysis	32
4.5	Discussion	34
5	Conclusion	36
	Bibliography	37

List of Figures

1.1	A high-level overview of the proposed research	2
3.1	A detailed architecture of the DiRLU framework, where the teacher transfers knowledge to the student via knowledge distillation, integrated with post-hoc feature unlearning module for data privacy also reintroduce unlearned features for scalability	10
3.2	Comparison of before and after applying SMOTE	12
3.3	Actor-Critic model architecture	14
3.4	Architecture of knowledge distillation (KD) framework	16
3.5	Architecture of feature unlearning process	19
4.1	Training accuracy and loss at each epoch of Teacher model	26
4.2	Confusion matrix of Teacher model	26
4.3	Training accuracy and loss at each epoch of Student model	27
4.4	Confusion matrix of Student model	27
4.5	Comparison of Teacher and Teacher unlearn models: accuracy, loss and f1 score	28
4.6	Confusion matrix of Teacher unlearn model	28
4.7	Comparison of Student and Student unlearn models: accuracy, loss and f1 score	29
4.8	Confusion matrix of Student unlearn model	29
4.9	Confusion matrices for all three stages: before removing the feature, after removing the feature, and after restoring the feature	30
4.10	XAI interpretation of the Teacher model's prediction for an attack case	30
4.11	XAI interpretation of top 12 features impacts for the Teacher model .	31
4.12	XAI interpretation of the Student model's prediction for an attack case	31
4.13	XAI interpretation of top 12 features impacts for the Student model .	31
4.14	XAI interpretation of the Teacher Unlearn model's prediction for an attack case	32
4.15	XAI interpretation of top 12 features impacts for the Teacher Unlearn	32

4.16 XAI interpretation of the Student Unlearn model's prediction for an attack case	33
4.17 XAI interpretation of top 12 features impacts for the Student Unlearn	33

List of Tables

2.1	Summary of relevant benchmark studies and comparison	8
3.1	Mathematical Symbols for Proposed Architecture	11
3.2	Different cyber-attacks listed in the Bot-IoT dataset	12
3.3	Feature description of the Bot-IoT dataset	13
3.4	Hyperparameters used in teacher model	17
3.5	Hyperparameters used in student model	19
4.1	Performance comparison of the Knowledge Distillation framework across three stages-(i) baseline with all features, (ii) after feature removal to simulate unlearning, and (iii) after feature restoration, utilizing A2C, Q-Learning, and Meta-RL	25

Chapter 1

Introduction

Robot Network, in short, Botnet, is a computer network infected by malware under the control of an attacking party known as Bot Herder [26]. Computers that are under the control of attackers are called bots. Attackers having remote access to computers through bots can read, update, and even decrypt sensitive data for financial gain or reputational loss. Controlling compromised devices over the networks remotely is a vital part for attackers. Bots are controlled directly or indirectly in two modes: Centralized client-server models and Decentralized peer-to-peer models. In 2016, Mirai botnet attack was on Dyn, an Internet performance management company. During the outage, the estimated losses were 22,000 US dollars per minute [18]. Modern malware, ransomware, and botnets pose serious cybersecurity risks to individuals, businesses, and governments.

Malicious content implants individuals' or organizations' devices in diverse ways, often exploiting human error and system vulnerabilities. Attackers typically start with phishing emails or websites, misleading victims into clicking malicious hyperlinks or downloading infected attachments. Once inside, botnets can spread across networks, gaining control of devices to carry out intended vicious activities. These infected systems are then used to deploy ransomware. Ransomware is malicious software or program that encrypts a victim's data, making it inaccessible until a ransom is paid to the attacker. Nevertheless, ransomware attacks can lead to a significant service outage for an organization. Attackers send commands to bots for launching attacks such as DDoS, Reconnaissance, Information Theft, Service Scan, Keylogging, etc.

When cyber threats such as DDoS, reconnaissance, information theft, or service scanning infiltrate an individual's or organization's security, they can impose significant damage over time if not noticed promptly. A distributed denial of service (DDoS) attack can immediately hinder services, rendering websites, vital tools, or devices unreachable and halting routine operations. Reconnaissance helps attackers to traverse the local network and discover weak and sensitive systems. Again, Information theft targets data such as client information or credentials, posing a risk and incurring reputational damage. Another type of attack, service scans, assist attackers in locating entry points into an organization's systems, which they then exploit to spread malware or secure unauthorized access. Keylogging secretly captures somebody's keystrokes, capturing confidential information and letting attackers control accounts or steal confidential data. Over time, these attacks seize

control, disrupt operations, and expose the organization to wider breaches.

1.1 Problem Statement

With the rapid growth of IoT devices, such as IP cameras, mobile phones, and smart home systems, ensuring network security has become increasingly challenging. Most devices lack built-in mechanisms to filter botnet traffic, making them highly vulnerable to attacks. Existing AI-based detection methods improve accuracy but are often too heavy for resource-limited IoT environments. Moreover, these models cannot “unlearn” sensitive or outdated features, which limits adaptability and raises privacy concerns like GDPR standards (right to be forgotten). However, retraining entire models from scratch is inefficient, impractical, also costly. Hence, there is a critical need for a lightweight, adaptive framework that can effectively detect botnets while selectively unlearning sensitive or outdated features without requiring retraining. This motivates the development of DiRLU, a reinforcement learning-based solution that ensures both efficiency and privacy in IoT security.

1.2 Thesis Objective

To protect an individual’s or organization’s sensitive data or reputation, we have to build and implement such security that can be adopted by any network device and is capable of defending against attacks before any botnet beaches inside. In this research, we developed a customized reinforcement learning based compact model named DiRLU that can detect and identify botnet attacks in the network layer. The model was trained on a big dataset where attack network traffic dominated normal traffic, which mimics realistic cyberattack scenarios in real-time. Further, to have a trustworthy model, we employed XAI to explain the model’s inside mechanism. Additionally, we demonstrated the feature un-learning process to maintain data security & privacy. Figure-1.1 illustrates the high-level architecture of the research. This research’s influential and significant contributions are summarized as follows:

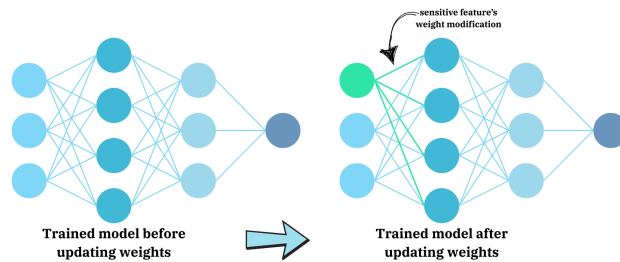


Figure 1.1: A high-level overview of the proposed research

- We propose DiRLU, a lightweight yet effective reinforcement learning framework using knowledge distillation to transfer knowledge from a large teacher model to a smaller student model. The student model is optimized with fewer hyperparameters, making it suitable for deployment on edge devices, such as mobile phones, surveillance cameras, and security gateways.

- To maintain user privacy and ensure model adaptability, we implement feature unlearning to remove sensitive or outdated attributes from the trained model without full retraining. This promotes GDPR compliance [24] and makes the model scalable.
- We reintroduce unlearned features to validate the effectiveness of the unlearning process. This comparison helps evaluate the impact of feature removal on model performance, ensuring the system remains robust and extendable.
- While benchmark models typically use only 5% of the dataset, we utilize a more diverse and representative 25% subset of the Bot-IoT dataset. This ensures better class coverage, improved generalization, and more reliable detection performance, especially in handling rare attack patterns.
- Explainable AI (XAI) techniques, such as LIME, are integrated to visualize and interpret model predictions. This enhances trust by demonstrating the reasoning behind attack classifications and helping cybersecurity analysts in decision-making.

1.3 Thesis Outline

The rest of the paper has been organized as follows: Chapter-2 explains some relevant research along with their contributions and limitations. Chapter-3 describes the methodology, dataset and algorithms used in this research in detail. Chapter-4 represents the results we obtained from this study, including XAI, to ensure the transparency of this research. We also describe some limitations and future goals of this paper. Finally, in Chapter-5, we included a consultation, providing the study's findings and potential areas for further exploration.

Chapter 2

Related Work

Several related studies have already been conducted to detect Botnet network traffic. This section briefly summarizes some studies and their approaches to detecting Botnet network traffic.

Leevy et al. [10] build a successful predictive machine learning model for detecting information theft attacks. This study used eight ML classifiers (four ensembles and four non-ensembles) and two evaluation metrics (AUC and AUPRC). The authors used only six features of the Bot-IoT dataset to get only specific insights about information theft attacks. The authors used 556 instances and other cutting-edge technologies, such as k-fold cross-validation, for training the model. Based on AUC and AUPRC results, the ensemble classifiers performed better than the non-ensemble classifiers. However, the LightGBM ensemble classifier performed top among all; it exhibited AUC results of 98% and AUPRC of 99%.

Koroniotis et al. [5] proposed a dataset, Bot-IOT, to simulate different types of network attacks, such as probing attacks, DDoS, DoS, and Information theft. Authors build the dataset to address other datasets' drawbacks with accurate labeling and incorporate recent attacks. They captured one GByte raw packet to extract complete information on network features. They introduced 32 features and created ten other hybrid features for simulation and testing the dataset's accuracy. They also evaluated the reliability of the dataset using ML (SVM) and deep learning algorithms (RNN and LSTM). SVM performed best with a full-featured dataset and scored 99% accurately, whereas RNN and LSTM performed 99% accurately with the ten best features only.

Alosaimi et al. [17] developed a machine learning-based intrusion detection system to secure IOT applications. The authors used the BOT-IoT dataset and a combination of deep learning and machine learning algorithms to train and test the model. The authors only used four files of the dataset, containing only 5-6% of the dataset. The first file contains only a DoS attack, the second one includes a DoS and DDoS attack, The third one contains two DDoS attacks, and The fourth one contains many attacks. They evaluated the results using accuracy, error rate, recall, and specificity. However, in the second database, the ensemble bag algorithm acquired an accuracy of 100% at all levels of the database, while the decision tree obtained an accuracy of 100% at all levels of the database in the third database. This study demonstrates remarkable improvements in intrusion detection, especially DoS attacks, compared

to existing models.

Farrokhmanesh et al. [2] designed a new machine learning-based model for detecting malicious activity in computers and networks. The authors proposed their model, which is based on audio signal processing techniques. They represented binary bytes as audio signals and applied MIR techniques for detecting malware. The authors used 3400 samples containing both malware and benign, however, they used the VX Heavens dataset for malware and Windows XP files for benign. They used KNN, AdaBoost, and Random Forest classifiers, among them AdaBoost performed best in terms of accuracy (92.2%). The authors demonstrated a lightweight novel model with good accuracy and is computationally efficient.

Bensaoud et al. [22] arranged a survey to examine how deep learning (DL) detects malware across multiple operating systems, including MacOS, Windows, iOS, Android, and Linux. They investigate different Deep Learning techniques, including text and image classification, pre-trained models, and multi-task learning, to determine how well they detect malware software. The paper also highlights significant problems, such as defined benchmark datasets, the complexity of deep learning models, and their sensitivity to adversarial attacks. It also emphasizes the value of Explainable AI (XAI) in making these models more transparent and intelligible. This study provides valuable insights into malware detection techniques by testing eight deep-learning approaches on various datasets.

Wang et al. [28] discuss the concept of machine unlearning, which focuses on enabling a trained model to remove or “forget” certain data. Their survey reviews different approaches, ranging from straightforward methods like deleting data and retraining, to more sophisticated gradient-based techniques that adjust the model without full retraining. They compare these strategies in terms of efficiency, computational cost, and impact on model accuracy. The study also highlights the challenges of ensuring that unlearning is applied reliably while still maintaining overall model performance. It identifies gaps in existing research, such as the need for consistent benchmarks. This paper proposes future ideas for increasing the scalability and usability of unlearning in large-scale machine-learning applications.

Miao et al. [25] propose DistillMal, a novel strategy for improving malware detection efficiency via a knowledge distillation architecture. The basic concept is to train a compact student network to replicate the performance of a more extensive, complex teacher network, resulting in a lightweight model suitable for deploying devices with limited resources. Extensive testing on two new datasets demonstrated that the student network performs similarly to the teacher model. The student model obtained 94.5% detection accuracy, compared to 95% for the teacher model, while lowering the number of parameters by 60% and the inference time by 50%. This demonstrates that DistillMal efficiently balances model size and detection accuracy. By leveraging knowledge distillation, the proposed method addresses the challenges of deploying large-scale deep learning models in environments with limited computational resources, such as mobile devices or IoT systems.

Galli et al. [23], in their work “Explainability in AI-based behavioral malware detection systems,” highlight the growing need for transparency in machine learning models that track behavior patterns to detect malicious software. To address this, they designed a detection model that incorporates interpretability tools such as SHapley

Additive exPlanations (SHAP), which help show how individual features influence the model’s decisions. This allows cybersecurity professionals to see which behavioral indicators play the most important role in classifying software as malicious or benign. Their approach achieved high accuracy, demonstrating strong malware detection performance. By combining explainability with detection, the system not only improves threat response but also builds trust and supports regulatory requirements. The study underscores the importance of striking a balance between powerful detection capabilities and clear, understandable AI-driven insights.

Kumar et al. [16] introduced EDIMA, a lightweight approach designed for early detection of IoT botnets at home network edge gateways. The framework combines traffic classification with Autocorrelation Function (ACF)-based testing to spot signs of bot activity. Their results show that EDIMA can accurately identify bot scanning and command-and-control (CnC) traffic, achieving a detection rate of 98% while maintaining a low rate of false positives. As the number of IoT devices rises, their performance stays strong. When authors use a Raspberry Pi, EDIMA exhibits short detection delays and low resource usage. The technology works better than current botnet-related traffic detection techniques. Furthermore, there is no appreciable processing penalty when EDIMA is implemented in real-time on edge devices. The technique provides scalable and efficient security against botnet threats in IoT environments.

To effectively identify Android malware and provide results that can be explained, the paper "PAIRED: An Explainable Lightweight Android Malware Detection System" introduces PAIRED [13]. To find malicious programs, PAIRED combines machine learning and static analysis methods. The system’s ability to differentiate between dangerous and benign programs is demonstrated by its 94% detection accuracy. Furthermore, PAIRED is appropriate for deployment on devices with constrained computing capabilities due to its lightweight architecture, which guarantees that it uses few resources. By using explainable AI algorithms, consumers may comprehend the characteristics that go into a detection judgment. The increasing security threats brought on by the extensive use of Android smartphones are addressed by PAIRED, which provides both explainability and excellent detection rates.

Patsakis et al. [27] investigated the use of Large Language Models (LLMs) for deobfuscating malicious code, focusing on the Emotet malware campaign. Their study tested four different LLMs to see how well they could interpret and simplify obfuscated code. The results showed that, although the models struggled with more advanced obfuscation techniques, they were able to successfully recover parts of malicious payloads, reaching accuracy levels of up to 85%. These findings suggest that LLMs could become a valuable tool for malware analysis, though further refinement is needed before they can fully meet the challenges of modern cybersecurity threats.

Wu et al. [33] introduced a privacy-enhanced deep learning framework for botnet detection, with a strong emphasis on preventing privacy leakage. Their method trains a feature extractor that masks sensitive information in network traffic while still capturing features relevant to botnet behavior. To achieve this, they employ a privacy-adversarial algorithm based on mutual information, reducing the link between private data and the anonymized features. The framework also incorporates federated learning, which trains the feature extractor across multiple users, fur-

ther strengthening privacy by keeping data decentralized. For detection, network traffic is converted into grayscale images and analyzed using a combination of Self-Attention Networks, CNNs, and Bi-LSTM models. Evaluation on the ISCX-2014 and CTU-13 datasets showed strong results: 100% accuracy and F1-score on CTU-13, and 98.44% accuracy with a 98.95% F1-score on ISCX-2014 without federated learning. Although accuracy dropped slightly when federated learning was applied (96.88% accuracy and 97.92% F1), it provided substantial privacy benefits. Overall, the framework outperformed several state-of-the-art methods, striking an effective balance between strong privacy guarantees and high detection accuracy.

Upreti et al. [12] present a comprehensive review of how reinforcement learning (RL) and deep reinforcement learning (DRL) can be applied to enhance IoT security. It shows how RL can help learn the best security policies through interactions between an agent and its environment, even with little prior knowledge. The paper also breaks down major IoT threats like DoS or DDoS attacks, jamming, and spoofing and explains how RL-based methods can tackle each. Some examples include using multi-agent RL to defend against DDoS attacks, DRL for power control to resist jamming and Q-learning for adaptively detecting spoofing. In cyber-physical systems like smart grids and smart transportation, RL is also used to model attackers and defenders so threats can be detected and handled early. The survey shows that RL and DRL techniques are more adaptable, faster, and resilient than traditional methods. However, challenges remain, such as dealing with high-dimensional data, limited observability, coordination between multiple agents, and defending against attacks that target the RL system. The authors emphasize the need for more research to make these methods scalable and practical for real-world use.

Muhati et al. [11] proposes a new deep reinforcement learning-based network intrusion detection system (NIDS) using the Asynchronous Advantage Actor-Critic (A3C) algorithm to tackle modern network security challenges. The authors highlight that many AI-based NIDS struggle because they lack efficient, low-cost ways to collect and process network data, which limits adaptability to new threats. To address this, they use automated network scanning and open-source tools for better data collection. The core idea is an A3C-based NIDS that combines value prediction and policy learning to detect both known and unknown attacks in dynamic environments. The system is tested on three benchmark datasets and under simulated poisoning attacks for robustness. Large-scale simulations with 300,000 nodes mimic real-world conditions. Results show high accuracy of 98.68% and the lowest false alarm rate compared to similar methods. The model remains stable and reliable across different settings, proving the strength of anomaly-based detection for evolving threats.

Numerous studies have explored effective botnet traffic classification methods utilizing deep learning and AI frameworks. However, Table-2.1 presents a comparative overview of these research efforts, highlighting their contributions and limitations.

Table 2.1: Summary of relevant benchmark studies and comparison

Ref.	Dataset	Feature	Model	Unlearning	KD	Accuracy
[10]	Bot-IoT (5%; Normal & Information Theft classes)	37 features	CatBoost, LightGBM, XGBoost, Random Forest, Decision Tree, Logistic Regression, Naive Bayes, MLP	N/A	N/A	Best: LightGBM AUC=0.987, AUPRC=0.994 ($\approx$ 98.34% accuracy)
[5]	Bot-IoT (5% subset)	Original & generated (10-best features selected via correlation & joint entropy analysis)	SVM, RNN, LSTM	N/A	N/A	SVM (10-best): 88.37%, SVM (full): 99.99%, RNN (10-best/full): 99.74%, LSTM (10-best/full): 99.74%
[17]	Bot-IoT	35 features	Decision Tree, Ensemble Bag, KNN, Linear Discriminant, SVM	N/A	N/A	DT: 99.99%, Ensemble Bag: 100%, KNN: 99.98%, SVM: 99.99%, LD: 100%
[2]	Custom Malware/Benign Dataset (3400 samples; PE and non-PE)	34 features (26 from MFCC + Chromagram)	KNN, AdaBoostM1, Random Forest	N/A	N/A	KNN: 90.0%, AdaBoostM1: 92.2%, Random Forest: 92.0%
[22]	Maling Dataset (grayscale/RGB malware images)	Grayscale and RGB images (converted from malware binaries)	EfficientNet, Inception V4, Xception, CapsNet	N/A	N/A	EffNet: 95.64%, Inception V4: 95.98%, Xception: 89.50%, CapsNet: 88.64%
[25]	VirusShare	512 features (API sequence length)	MLP, TextCNN, Catak (BiLSTM), BERT-base, DistillBert, DistillMal	N/A	Teacher: BERT Student: TextCNN	MLP: 83.5% TextCNN: 88.4% Catak (BiLSTM): 66.3% BERT-base: 89.2% DistillBert: 89.0% DistillMal: 89.1%
[23]	Mal-API-2019	128 (sequence length)	LSTM, BiLSTM, GRU, Attention, MultiHeadAttention	N/A	N/A	LSTM: 47.53%, BiLSTM: 52.88%, GRU: 47.69%, Attention: 52.18%, MultiHeadAttention: 47.69%
[16]	IoT-BPR-NSS Testbed & UNSW IoT	6	Random Forest, SVM, Gaussian Naive Bayes	N/A	N/A	Testbed: RForest: 100%, SVM: 99%, GNB: 97% UNSW IoT: RForest: 100%, SVM: 91%, GNB: 92%
[13]	Drebin-215, Malgenome-215 & CICMalDroid2020	35	Random Forest, Logistic Regression, Decision Tree, Gaussian Naive Bayes, SVM	N/A	N/A	RF: Drebin-215: 98.07%, Malgenome-215: 98.73%, CICMalDroid2020: 97.98%
[27]	Emotet malware campaign (2,000 obfuscated PowerShell scripts; 2,869 URLs, 2,512 domains)	N/A (LLM prompt-based extraction)	GPT-4, Gemini Pro, Code Llama 34B Instruct, Mixtral 8x7B	N/A	N/A	URL extraction accuracy: GPT-4: 69.56%, Gemini Pro: 36.84%, Code Llama: 22.13%, Mixtral: 11.59%; Domain extraction accuracy: GPT-4: 88.78%, Gemini Pro: 55.99%, Code Llama: 35.46%, Mixtral: 24.92%
[33]	ISCX-2014 & CTU-13 botnet traffic datasets	Raw traffic bytes (image-based input)	CNN + Bi-LSTM with privacy-adversarial feature extractor trained via mutual information minimization & federated learning	N/A	N/A	CTU-13: 100% accuracy/F1 (both FL and non-FL) ISCX-2014: non-FL: Acc=98.44%, F1=98.95; FL: Acc=96.88%, F1=97.92
[11]	UNSW-NB15, AWID, NSL-KDD (combined; with 30% poisoning attack simulation)	Network traffic features and packet-level metadata	Asynchronous Advantage Actor-Critic (A3C) deep reinforcement learning (actor-critic)	N/A	N/A	Accuracy: 98.68%, Precision: 98.4%, Recall: 98.9%, FP rate: 1.59%
Di-RLU	Bot-IoT (25% & 30% subset)	34 features of network traffic	A2C, Q-Learning & Meta RL	Post-hoc weight modification	Teacher: A2C Student: A2C	Best: A2C (25% subset) Accuracy=99.60%, F1=99.80%

Chapter 3

Methodology

This research aims to identify attack traffic accurately, even when attack traffic dominates benign traffic, by utilizing a large-scale dataset. Besides, we seek to develop a compact model that performs efficiently without requiring extensive computational resources. Additionally, the study focuses on the ability to selectively forget sensitive data whenever needed, ensuring enhanced data security and privacy. Figure-3.1 illustrates the detailed architecture of the proposed model, DiRLU. The process begins with IoT devices generating network traffic data. This data then goes through preprocessing and feature extraction using engineering techniques. The processed data was split into three portions for training, testing, and validation. In the modeling stage, we use reinforcement learning methods such as A2C, Q-learning and Meta-RL. A feature unlearning module works with these RL models to selectively remove sensitive or outdated features from the system. To make the trained model lighter, we apply knowledge distillation. The distilled model then produces the output (attack or benign). Using XAI (LIME), we explain the model's decisions so they are transparent and trustworthy. Finally, performance metrics are used to evaluate the effectiveness of the models and to measure the effect of the model.

The following sections provide a detailed explanation of the various phases of the proposed framework. Since multiple symbols are used in later sections, a list of commonly used symbols is included in table-3.1 to enhance the clarity of this paper.

3.1 Data Description

The BoT-IoT publicly available dataset was built in a real network environment in the Cyber Range Lab of the University of New South Wales (UNSW) Canberra [6]. The environment was set in such a way that contained both normal and botnet network traffic. In the dataset, normal traffic is dominated by attack traffic. The dataset is heavily imbalanced, which reflects real-world cyberattack scenarios. They captured traffic using Wireshark, and the total size of pcap files is 69.3 GB, with more than 72 million records and 35 features. The Bot-IoT dataset includes different attacks such as DDoS, DoS, OS and Service Scan, Keylogging and Data exfiltration attacks, which are explained in Table-3.2.

In Table-3.3, we listed all populated features (total 31) along with their description.

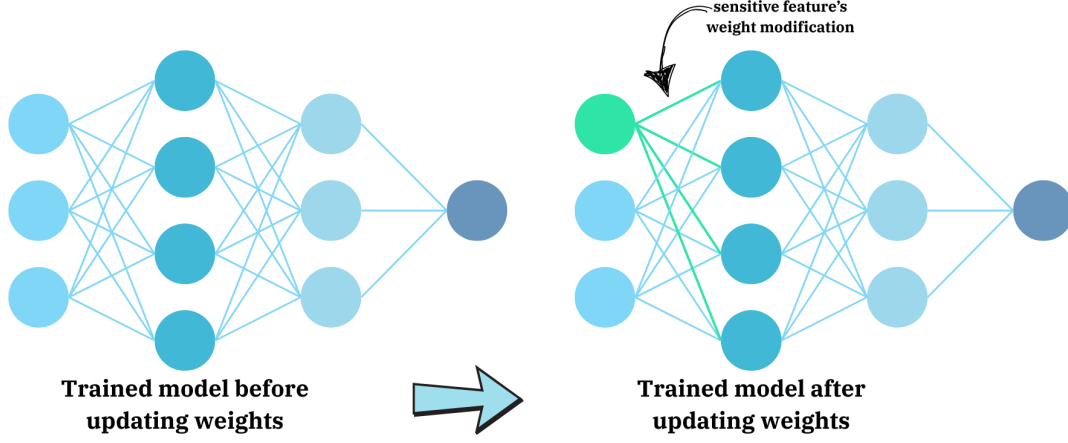


Figure 3.1: A detailed architecture of the DiRLU framework, where the teacher transfers knowledge to the student via knowledge distillation, integrated with post-hoc feature unlearning module for data privacy also reintroduce unlearned features for scalability

3.2 Data Pre-Processing

Data pre-processing is required to ensure that the data fit properly into the proposed model as it standardizes, normalizes, and handles missing values. This alignment optimizes the model performance and accuracy by furnishing data with uniform, high-quality, and ideal inputs. Firstly, we have dropped the *category* and *subcategory* column as we do not require them for classification. So, out of 35 features, we worked with 33 features; among them, we selected 32 features for further preprocessing. However, the column named *attack* is used for classification.

3.2.1 Feature Pruning

To have a refined and clean dataset for training the model, we removed all missing or nan values. Some of the features didn't even have any values except nan. So, after removing all nan values, we got 27 columns and 15,000,000 rows. The completely empty features are *smac*, *dmac*, *soui*, *doui*, *sco*, and *dco*.

3.2.2 Label Encoding

Label encoding is an essential technique for converting category input to numerical values, which makes it suitable for deep learning algorithms. For example, we transformed the "attack" category, assigning the value 1 for attack and 0 for not attack. We utilized the *LabelEncoder* function from the *sklearn* library. This conversion allows the model to process the categorical data and use them for classification purposes.

Table 3.1: Mathematical Symbols for Proposed Architecture

Symbol	Description
Normalization	
x	Original value of a feature
$x_{\min}$	Minimum value of a feature in the dataset
$x_{\max}$	Maximum value of a feature in the dataset
x'	Normalized value after Min-Max scaling
Actor Network	
$\pi_{\theta}(a s)$	Policy function of the actor network with parameters θ
θ	Parameters of the actor network
L_{actor}	Loss function for the actor network
A_t	Advantage function at time step t
α	Learning rate for the actor network
∇_{θ}	Gradient with respect to actor parameters
Critic Network	
$V_{\phi}(s_t)$	Value function of the critic network for state s_t
L_{critic}	Loss function for the critic network
R_t	Expected cumulative reward at time t
r_t	Immediate reward at time step t
γ	Discount factor in reinforcement learning
β	Learning rate for the critic network
Knowledge Distillation	
$P_T(i)$	Soft target probability from the teacher model for class i
z_i	Logit (output) from the teacher model for class i
T	Temperature parameter for softmax smoothing
$P_S(i)$	Student model's predicted probability for class i
y_i	Ground truth label for class i
L_{CE}	Cross-entropy loss for true labels
L_{KD}	Distillation loss (KL divergence with soft targets)
L	Total loss function
Evaluation	
y_i	Ground truth label for class i
$\hat{y}_i$	Predicted probability for class i by the model
n	The number of samples

3.3 Data Split

Splitting the dataset into a train, validation, and test subset is crucial for training, validating, and testing the model. We have taken 25% of the total dataset, whereas previous state-of-the-art research has taken only 5% [10]. We got 15,000,000 (Fifteen million) data points in 25% of the whole dataset. We use a random splitting technique to split the dataset into three parts: training, validating, and testing. We took 70% of data points for training, 10% for validating & tuning hyper-parameters, and another 20% for testing the model. We randomly choose the 70:20:10 (train:test:validation) ratio for splitting the dataset. However, after splitting, we got 10,500,000 (Ten million five hundred thousand) data points for training the model, 1,500,000 (One million five hundred thousand) data points for

Table 3.2: Different cyber-attacks listed in the Bot-IoT dataset

Attack Name	Description
Denial of Service (DoS)	An attempt to make a machine or network resource unavailable to its intended users
Distributed Denial of Service (DDoS)	Multiple compromised systems attack a target, causing denial of service for users
Reconnaissance	Activities aimed at gathering information about a network for planning future attacks
Information Theft	Unauthorized access and retrieval of sensitive information
Data Exfiltration	Unauthorized transfer of data from a computer or other device
OS and Service Scan	This technique reveals systems and open service (HTTP, FTP, or SMTP) vulnerabilities
Keylogging	This is a secret recording of a user's keystrokes to capture sensitive information

tuning hyper-parameters & validating the model, and 3,000,000 (Three million) data points for testing the model's performance.

3.4 Data Augmentation

We have taken 25% data points from the BOT-IOT dataset, which gives us a total of 15,000,000 rows. Among those rows, we found attack rows 14,992,430 (Fourteen million, nine hundred ninety-two thousand, four hundred thirty) and the remaining 7,570 (Seven thousand five hundred seventy) non-attack rows. The dataset we have chosen for this research is highly imbalanced, and the ratio of attack vs non-attack is 98:2. So, to balance the dataset, we have used a technique called SMOTE.

SMOTE (Synthetic Minority Oversampling Technique) is an oversampling preprocessing technique used to address class imbalances in the dataset [8]. It generates synthetic data points by selecting a minority class and creating new data points along the line segments, attaching them to their closest neighbors. This method introduces diversity in the artificial samples, helping to create a non-biased dataset. SMOTE has been widely adopted in machine learning applications to enhance classification performance in imbalanced datasets [1].

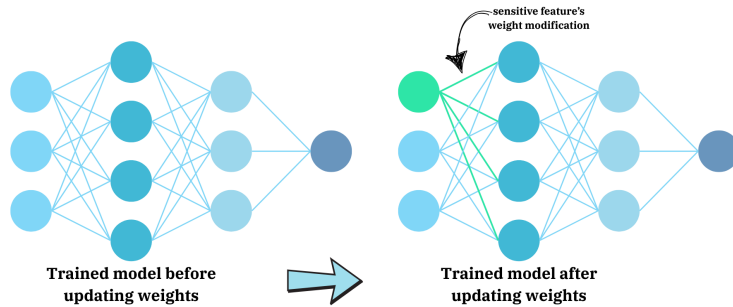


Figure 3.2: Comparison of before and after applying SMOTE

Table 3.3: Feature description of the Bot-IoT dataset

Feature Name	Feature Description
pkSeqID	Packet number
stime	Packet capture start time
flgs	Flow state flags seen in transactions
flgs_number	Numeric num. of flgs
proto	Protocol used in the flow
saddr	Source IP
sport	Source port
daddr	Destination IP
dport	Destination port
pkts	Total packet count
bytes	Total bytes in the flow
state	Packet state
state_number	Numeric num. of state
ltime	Packet capture last time
seq	Packet sequence number
dur	Total duration
min	Minimum duration of aggregated records
max	Maximum duration of aggregated records
mean	Average duration of total time
stddev	Standard deviation of all records
sum	Total duration of all records
spkts	Source to destination packet count
dpkts	Destination to source packet count
sbytes	Byte count of source to destination packets
dbytes	Byte count of destination to source packets
rate	Packets per second in the flow
srate	Packets per second from source to destination
drate	Packets per second from destination to source
attack	Label: 0 for Normal and 1 for Attack
category	Network traffic category
subcategory	Network traffic sub-category

After applying smote to the imbalance training dataset, we got 7,035,000 (Seven million thirty-five thousand) rows for the attack class (67%) and another 3,465,000 (Three million four hundred sixty-five thousand) rows for the non-attack class (33%), which makes the dataset balanced and non-biased. In Figure-3.2, we have represented the ratio and effect of before & after applying smote.

3.5 Data Normalization

Data normalization is a vital pre-processing technique required for scaling feature values. In the dataset, some feature values are large, and some are very small. Basically, it reshapes numerical values to fit into standard scale values. In this research, we applied deep learning techniques, so we used the Min-Max normalization technique to have values between 0 and 1. Recent studies have proved that Min-Max normalization can significantly improve the accuracy of deep learning models across various applications [15]. The Min-Max function normalized the data using

equation-3.1.

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (3.1)$$

In equation-3.1, x is the original value, $x_{\min}$ is the minimum & $x_{\max}$ is the maximum value in the dataset, and x' is the normalized value.

3.6 Reinforcement Learning

Reinforcement learning is a type of machine learning where the model learns by interacting with its environment and receiving rewards or penalties for each of its actions, similar to trial and error [19]. Unlike traditional deep learning, it does not rely on labeled data but improves through feedback. It is beneficial for solving complex problems like IoT Security [7].

We have used the actor-critic model because of its enhanced stability compared to traditional reinforcement learning approaches by combining policy optimization with value estimation. In the Fig-3.3, we represented a simple architecture of the actor-critic model. The actor-critic framework operates through two interconnected neural networks that collaborate to optimize decision-making processes. The actor-network functions as a policy generator, determining which actions to execute based on current states. In contrast, the critic network serves as an evaluator, assessing the quality and long-term value of the actor's chosen actions. During training, the actor adjusts its policy parameters using feedback signals provided by the critic, which simultaneously refines its value estimations based on observed rewards and feedback. This mechanism creates a balanced learning environment by exploring new strategies. The temporal difference error, calculated as the dissimilarity between predicted and actual rewards, is the primary learning signal that guides both networks toward optimal performance [20]. The actor updates its policy through gradient ascent on expected rewards, while the critic minimizes prediction errors through standard value function approximation techniques. This iterative process continues until convergence, resulting in a robust policy that maximizes cumulative rewards while maintaining stable learning dynamics throughout the training process.

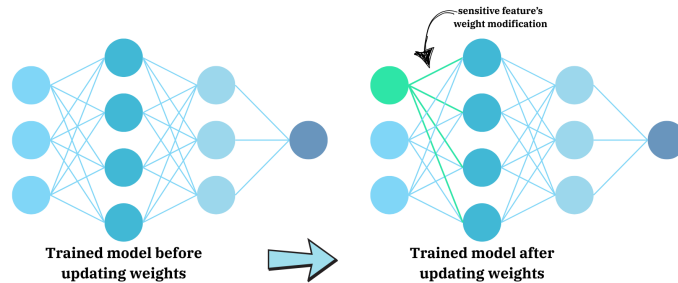


Figure 3.3: Actor-Critic model architecture

From equations 3.2-3.7, we have described the mathematical formulas of the actor-critic model, including both forward propagation and backpropagation update rules.

$$\pi_{\theta}(a|s) = P(a|s; \theta) \quad (3.2)$$

In equation-3.2 is Actor Policy Function is shown where the probability of taking action a in state s , and θ denotes the actor network parameters.

$$L_{actor} = -\mathbb{E}[\log \pi_{\theta}(a_t|s_t) \cdot A_t] \quad (3.3)$$

In equation-3.3 shows the actor loss function, where A_t is the advantage function and $\mathbb{E}[\cdot]$ represents the expected value operator.

$$V_{\phi}(s_t) = \mathbb{E}[R_t|s_t] \quad (3.4)$$

In equation-3.4 shows Critic Value Function where critic parameters is ϕ , and R_t denotes the expected cumulative reward.

$$L_{critic} = \mathbb{E}[(r_t + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t))^2] \quad (3.5)$$

In equation-3.5, the critic loss function is shown, where $V_{\phi}(s_t)$ is the state value function, γ represents the discount factor, and r_t is the immediate reward at time step t .

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \cdot A_t \quad (3.6)$$

In equation-3.6 presents the actor parameter update rule, where α is the learning rate and ∇_{θ} represents the gradient with respect to actor parameters.

$$\phi \leftarrow \phi - \beta \nabla_{\phi} L_{critic} \quad (3.7)$$

In equation-3.7, the critic parameter update is shown, where β is the critic learning rate and ∇_{ϕ} denotes the gradient with respect to critic parameters.

We used actor-critic reinforcement learning in a teacher-student framework for knowledge distillation. In this design, the teacher model has two outputs: the actor makes binary classification decisions using a sigmoid function, and the critic provides continuous value estimates with a linear function. We added an attention mechanism that learns to focus on all the input features with an entropy regularization. We also implemented shared hidden layers, L2 and dropout regularization for better generalization. We use weighted loss functions to handle class imbalance so that underrepresented classes get fair attention during training. Overall, the model jointly optimizes three goals: accurate classification by the actor, precise value estimation by the critic, and balanced attention. This multi-objective setup helps the teacher learn rich representations, which are then passed on to the student model for better learning.

3.7 Knowledge Distillation

Knowledge Distillation (KD) is a model compression technique where a large, complex teacher model transfers its knowledge to a smaller, more efficient student model without significant loss in performance. In this method, the teacher model is initially trained on a dataset, and its output, such as soft probabilities (logits) or intermediate feature representations, is used to assist the student model's training. The

student model is optimized using a distillation loss function, which combines standard loss (cross-entropy) with a term encouraging the student to mimic the teacher’s outputs. The benefit of the KD is that it reduces the size of networks while maintaining high accuracy and faster inference, making it appropriate for deployment on resource-constrained devices such as mobile phones and edge computing platforms. This technique has been widely adopted in security applications, improving the robustness of AI models against adversarial attacks while maintaining efficiency [9]. We have used KD to classify malware and benign traffic with low resources. In Fig-3.4, we have included an architecture diagram of the knowledge distillation model.

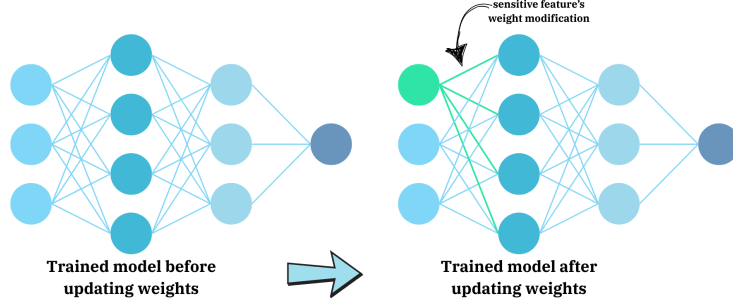


Figure 3.4: Architecture of knowledge distillation (KD) framework

3.7.1 Teacher Model

We implemented a teacher model within a Knowledge Distillation (KD) framework, where a larger Actor-Critic model transfers its knowledge to a smaller one (student model). The teacher model learns to map input data to output probabilities, which are then used to transfer knowledge to a smaller one. The teacher model produces soft targets P_T by applying a temperature-scaled softmax function to the logits:

$$P_T(i) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \quad (3.8)$$

In equation-3.8, z_i is the logit (output) from the teacher model and T is the temperature hyperparameter, which controls the smoothness of the probability distribution. Higher T produces smoother probability distributions.

The teacher model is an attention-enhanced actor-critic architecture. It includes a feature-level attention mechanism that learns from all the individual input features. The attended representation is concatenated with the original input and passed through three shared dense layers with ReLU activation and L2 regularization. The actor’s head generates probability scores using a sigmoid activation, while the critic’s head estimates value functions using a linear activation. An additional entropy penalty regularizes the attention weights to prevent overly confident focus on a few features. Class weights are dynamically computed and incorporated into the custom binary cross-entropy loss function to address class imbalance. Furthermore, the model uses early stopping on validation accuracy to avoid overfitting and restore the best-performing weights. Training continues until early stopping is triggered, typically within 25–30 epochs. Table-3.4 provides a complete list of

the hyperparameters and their corresponding values used to improve the teacher model’s performance.

Table 3.4: Hyperparameters used in teacher model

Hyperparameter	Value
Model	Actor-Critic
Number of Hidden Layers	3
Activation Function (Hidden Layers)	ReLU
Activation Function (Actor Output)	Sigmoid
Activation Function (Critic Output)	Linear
Regularization	L2 & Entropy Penalty
Actor Loss Function	Weighted BSE
Critic Loss Function	MSE
Optimizer	Adam
Epoch Number	Early stopping (max 30)
Batch Size	512
Evaluation Metric	Accuracy & F1 Score

To make predictions, probability scores from the actor’s head are binarized using a 0.8 threshold, and the F1 score is calculated to evaluate classification performance. This trained teacher model is the foundation for knowledge transfer, guiding the development of a smaller, more efficient student model with similar decision boundaries and attention-driven behaviour.

3.7.2 Student Model

A student model is a smaller, more efficient Actor-Critic model trained to replicate the behaviour of a larger and more complex teacher model. It learns using the teacher’s softened predictions, allowing it to perform well while being lightweight and faster. With appropriate distillation techniques, student models can accurately mimic the behaviour of the teacher model while drastically reducing the model size and computational complexity, making them ideal for real-time applications in mobile and edge computing [14]. The student model mimics the teacher’s behavior by optimizing a total loss that combines standard cross-entropy (for true labels) and distillation loss (KL divergence with the teacher’s soft targets).

The Cross-Entropy Loss for true labels (L_{CE}),

$$L_{CE} = - \sum_i y_i \log P_S(i) \quad (3.9)$$

In equation-3.9, y_i is the ground truth label and $P_S(i)$ is the student model’s predicted probability.

Distillation Loss (KL divergence with soft targets) (L_{KD}),

$$L_{KD} = T^2 \sum_i P_T(i) \log \frac{P_T(i)}{P_S(i)} \quad (3.10)$$

In equation-3.10, T is temperature parameter which controls the smoothness of the probability distribution, $P_T(i)$ is probability output from the teacher model of class i and $P_S(i)$ is probability output from the student model of class i .

The total loss function (L),

$$L = \alpha L_{\text{CE}} + (1 - \alpha) L_{\text{KD}} \quad (3.11)$$

In equation-3.11, α is a weighting factor that balances the cross-entropy loss L_{CE} and the distillation loss L_{KD} .

In this research, the student model consists of a smaller actor-critic model. This student model is smaller and faster than the teacher model. The actor helps predicts the final class (benign or attack), and the critic, which gives a value or score to help guide learning.

We use an attention mechanism to make the model focus on all the input features. These attention-weighted inputs are combined with the original input and passed through two hidden layers. Each layer uses the ReLU activation function, and we apply L2 regularization to reduce overfitting. The final part of the model has two outputs: the actor output uses a sigmoid function for classification, and the critic output uses a linear function to estimate value. We also apply an entropy penalty on the attention weights to ensure the model doesn't focus too much on only a few features. We train the model using the Adam optimizer with a batch size 32. The actor output is trained with a custom loss function that combines two types of loss: binary cross-entropy, which checks how well the predictions match the actual labels, and KL divergence, which helps the student learn from the soft outputs of the teacher model. We use a temperature value of 2.0 to smooth the teacher's predictions and an alpha value 0.5 to balance the two losses. For the critic output, we use mean squared error with class weights to handle imbalance in the data. We also include the entropy penalty in the total loss to balance the attention distribution. We use early stopping based on validation accuracy to avoid overfitting, which stops training when the model stops improving. Table-3.5 provides the list of the hyperparameters and their corresponding values used to improve the student model's performance.

3.8 Un-learning

Feature unlearning is a technique designed to remove the influence of specific features from a trained model, ensuring that once removed, those features no longer affect the model's outputs. The Price of Forgetting [29] introduced an economic approach by offering payments to users for keeping their data, helping servers reduce costs while still respecting user privacy. While our approach plays an important role in meeting privacy regulations, mitigating security risks (such as eliminating data), and enabling efficient model updates without the need for full retraining. Key benefits include regulatory compliance by securely removing sensitive data, cost and time savings from avoiding comprehensive retraining, and greater transparency and trust in AI systems. Recent developments in feature unlearning have introduced methods

Table 3.5: Hyperparameters used in student model

Hyperparameter	Value
Model	Actor-Critic
Number of Hidden Layers	2
Activation Function (Hidden Layers)	ReLU
Activation Function (Actor Output)	Sigmoid
Activation Function (Critic Output)	Linear
Regularization	L2 & Entropy Penalty
Actor Loss Function	Distillation Loss (CE+KL)
Critic Loss Function	Weighted MSE
Temperature	2.0
Alpha	0.5
Optimizer	Adam
Epoch Number	Early stopping (max 30)
Batch Size	512
Evaluation Metric	Accuracy & F1 Score

such as data pruning and selective retraining, which allow models to effectively erase the impact of targeted features while preserving overall performance [21]. In practice, this balance of privacy, efficiency, and model reliability is critical for real-world applications.

The post hoc weight modification technique is used to "forget" sensitive data/features for data security. Instead of retraining the whole model, this technique sets zero weight in the first layer of the specific feature. Post hoc weight modification (PHWM) allows the model to forget specific features by modifying a single weight in the network without costly and time-consuming retraining [30]. In Fig-3.5, we illustrate the process of unlearning.

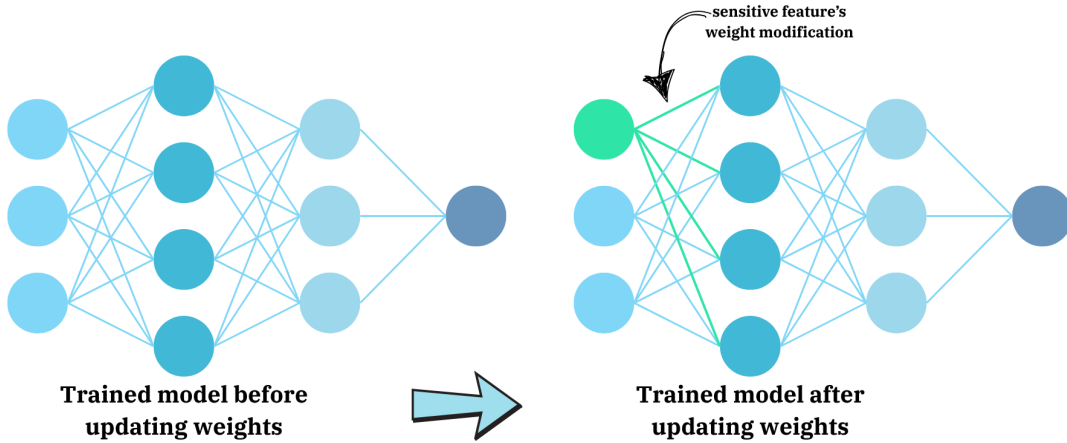


Figure 3.5: Architecture of feature unlearning process

3.8.1 Teacher Unlearn

The procedure of teacher unlearning involves selectively dismissing the second top feature from the dataset that has the most impact on the teacher model. We have used XAI to identify the feature. In this experiment, we have found the second top

feature, 'flgs', for removal from the pre-trained teacher. We removed the feature from the input datasets to ensure the model unlearned them. We also set their corresponding weights in the first layer of the network to zero, preventing them from influencing predictions. After modifying the weight matrix, we reloaded it into the model to ensure the feature was no longer used. To measure the impact of these changes, we re-evaluated the model using accuracy and F1-score. This approach helps improve model interpretability by removing redundant or sensitive features and keeping computations efficient. The approach ensures that the model no longer depends on the removed features, enforcing unlearning at both the data and model levels.

3.8.2 Student Unlearn

The process of student unlearning involves carefully removing the second most influential feature from the dataset that affects the student model. We used XAI to specify the features; in this case, the feature was 'flgs'. The feature was removed from the input datasets to ensure the student model no longer learned or made predictions using them. We set the corresponding weights of the feature in the network's first layer to zero, ensuring the model couldn't rely on them for predictions. After making these changes to the weight matrix, the feature was refilled into the student model to confirm that they were no longer in use. To understand how this influenced the model, we re-evaluated it using accuracy and F1-score. This approach improves the model's interpretability by removing unnecessary or sensitive features while maintaining efficient computations.

3.9 Explainable Artificial Intelligence (XAI)

Explainable AI (XAI) in machine learning and deep learning focuses on making model decisions more transparent and easier to understand. It demonstrates how models arrive at their predictions, which helps build trust and ensures fairness. XAI is also valuable for assessing feature importance by showing how different factors influence model outcomes, an essential step for tuning models and improving performance. By improving interpretability, XAI supports accountability and strengthens the reliability of AI systems. Common approaches include local and global interpretability methods such as SHAP and LIME, which provide insights into both individual predictions and overall model behavior, making it easier to detect biases or errors in decision-making [3]. Recent work has also applied XAI to anomaly detection in NFV systems, where masked autoencoders are combined with explainability techniques to enhance interpretability and support effective fault localization [31].

We used LIME (Local Interpretable Model-agnostic Explanations) to understand how the model makes decisions. LIME explains predictions by generating minor variations around a given data point and analyzing how the model responds. LIME has been applied in diverse machine learning tasks, enhancing model transparency and helping researchers identify potential biases or errors in model predictions [4]. A single test instance was randomly chosen, and LIME ranked the top 12 features that influenced the model's conclusion. The explanation mode was set to classifica-

tion, and discretization was turned off to retain continuous feature representations. The Results were visualized interactively using `show_in_notebook()`. Using LIME, we gained valuable insights into which features played a key role in the model’s classification.

3.10 Model Evaluation

In this research, we tested the model’s performance using various commonly used metrics, such as accuracy, F1 score, and loss.

Regardless of class, accuracy determines the overall accuracy of the model’s predictions and evaluates its performance.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (3.12)$$

Precision measures the percentage of the model’s predictions that are correct, and recall measures the percentage of relevant data points that the model correctly identified. The mean of precision and recall is the F1 score.

$$Precision = \frac{TP}{TP + FP} \quad (3.13)$$

$$Recall = \frac{TP}{TP + FN} \quad (3.14)$$

$$F1 \text{ score} = \frac{2 * Precision * Recall}{Precision + Recall} \quad (3.15)$$

In equations 3.12-3.15, TP denotes true positives, TN denotes true negatives, FP denotes false positives, and FN denotes false negatives.

Another evaluation metric, Binary Cross-Entropy (BCE) is a loss function used in this paper for binary classification. It measures the difference between accurate labels (y) and predicted probabilities ($\hat{y}$). It penalizes inaccurate predictions more when predictions are confident but wrong.

$$BCE = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (3.16)$$

In the equation 3.16, y_i is the actual label, $\hat{y}_i$ is the predicted probability, and n is the number of samples.

3.11 Proposed Algorithm

In algorithm-1, we explain the pseudocode for the proposed model’s architecture in detail. We built the model using Python (v3.12) and different libraries, such as tensorflow, pandas, lime, matplotlib, etc., on Visual Studio Code IDE.

Algorithm 1 Pseudocode of DiRLU

```
1 function PREPROCESS(df)
2    $df_c \leftarrow \text{CLEAN}(df)$ ;  $df_b \leftarrow \text{APPLYSMOTE}(df_c)$ ;  $df_{tr}, df_{val}, df_{te} \leftarrow \text{SPLIT}(df_b)$ 
3   return  $df_{tr}, df_{val}, df_{te}$ 

4 function TRAINA2CTEACHER(df_train)
5    $teacher \leftarrow \text{A2C}()$ ;  $w \leftarrow \text{COMPCLSWEIGHTS}(df\_train)$ 
6   while NOT CONVERGED(teacher, EarlyStop) do
7      $(x, y) \leftarrow \text{SAMPLE}(df\_train)$ 
8      $L \leftarrow \text{WEIGHTEDBCE}(\text{actor}(x), y, \text{cls\_weights}) + \text{MSE}(\text{critic}(x), y)$ 
        $+ \lambda_{attn} \cdot \text{ENTROPY}(\text{attention})$ 
9      $\text{UPDATEA2C}(teacher, \nabla L, \text{Adam})$ 
10  return teacher

11 function FEATUREUNLEARN(model, forgetFeat)
12   $(W_u, b_u) \leftarrow \text{model.GETWEIGHTS}(\text{first\_dense}, \text{INDEX}(\text{forgetFeat}))$ 
13   $W_v \leftarrow 0.0$ ;  $b_v \leftarrow 0.0$ 
14   $\text{model} \leftarrow \text{model.UPDATEWEIGHTS}(\text{first\_dense}, \text{INDEX}(\text{forgetFeat}), W_v, b_v)$ 
15  return model

16 function DISTILLTOSTUDENT(teacher, df_train, T)
17   $student \leftarrow \text{A2C}()$ ;  $\text{cls\_weights} \leftarrow \text{COMCLSWEIGHTS}(df\_train)$ 
18   $t\_probs \leftarrow \text{SOFTMAX}(\text{teacher}(df\_train)/T)$ 
19  while NOT CONVERGED(student, EarlyStop) do
20     $(x, y) \leftarrow \text{SAMPLE}(df\_train)$ 
21     $L \leftarrow \alpha \cdot \text{CE}(\text{student}(x), y) + (1 - \alpha) \cdot T^2 \cdot \text{KL}(\text{student}(x)/T, t\_probs)$ 
       $+ \text{MSE}(\text{critic}(x), y) + \lambda_{attn} \cdot \text{ENTROPY}(\text{attention})$ 
22     $\text{STEP}(\text{student}, \nabla L, \text{Adam})$ 
23  return student

24 function RESTOREFEATURES(model, forgetFeat)
25   $(W_v, b_v) \leftarrow \text{SAVEDWEIGHTSBIASES}(\text{first\_dense}, \text{INDEX}(\text{forgetFeat}))$ 
26   $\text{model} \leftarrow \text{model.UPDATEWEIGHTS}(\text{first\_dense}, \text{INDEX}(\text{forgetFeat}), W_v, b_v)$ 
27  return model

28  $df_{tr}, df_{val}, df_{te} \leftarrow \text{PREPROCESS}(df)$ ;  $teacher \leftarrow \text{TRAINA2CTEACHER}(df_{tr})$ 
29  $student \leftarrow \text{DISTILLTOSTUDENT}(teacher, df_{tr}, T)$ 
30  $\text{base\_models}(\mathcal{B}) \leftarrow [\text{teacher}, \text{student}]$ 
31  $\text{unlearn\_models}(\mathcal{UL}) \leftarrow []$ 
32  $\text{restore\_models}(\mathcal{RE}) \leftarrow []$ 
33 for all  $m \in \text{base\_models}$  do
34    $\text{unlearn\_models} \leftarrow \text{FEATUREUNLEARN}(m, df_{fg})$ 
35 for all  $m_{ul} \in \text{unlearn\_models}$  do
36    $\text{restore\_models} \leftarrow \text{RESTOREFEATURES}(m_{ul}, df_{fg})$ 
37  $\text{all\_models} \leftarrow \mathcal{B} \cup \mathcal{UL} \cup \mathcal{RE}$ 
38 for all  $x \in df_{te}$  do
39   for all  $m \in \text{all\_models}$  do
40      $\hat{y} \leftarrow \text{PREDICT}(m, x)$ 
41      $\text{EXPLAINWITHLIME}(\hat{y})$ 
42  $\text{COMPARERESULTS}(\text{all\_models})$ 
```

To conduct the research, a high-performance workstation was configured with 64GB

of RAM, a 3.40GHz processor, and an NVIDIA 12GB GPU running Windows 11. This setup ensured optimal processing and memory capacity for data-intensive tasks.

Chapter 4

Result and Analysis

This section includes a detailed explanation of the results (only 25% datapoints) obtained using the algorithms and dataset described in the methodology section. To ensure transparency, we used XAI techniques like LIME to explain how each model makes predictions and included two sample test case scenarios. Finally, we will evaluate the study’s shortcomings and explore potential areas for improvement.

Table-4.1 summarizes the Knowledge Distillation framework’s performance across three stages: before feature removal, after feature removal, and after feature restoration. We conducted an evaluation based on accuracy, F1 Score, and loss for A2C, Q-Learning, and Meta-RL algorithms on 25% and 30% data subsets, comparing both Teacher and Student models. Before feature removal, Meta-RL delivers the best Teacher performance, achieving 99.704% accuracy and 99.852% F1 on the 30% subset. For Student models, A2C leads with 99.602% accuracy and 99.800% F1 on the 25% subset. Q-Learning performs well but slightly trails the other two. After feature removal, performance drops for some algorithms, particularly Meta-RL on the 30% subset, where Teacher’s accuracy drops to 85.336% and Student accuracy to 79.471%, showing its sensitivity to missing features. A2C remains relatively stable, with Student accuracy at 99.354% (25% subset) and 93.163% (30% subset). Q-Learning experiences moderate drops, with Student accuracy falling to 98.109% on the 25% subset. After feature restoration, most models regain near-original performance. Meta-RL Teacher scores return to 99.704% accuracy and 99.852% F1, while A2C Student scores hit 99.603% accuracy and 99.801% F1 on the 25% subset. Overall, A2C proves the most stable across all stages, while Meta-RL performed better for the Teacher model only, particularly with limited datapoints.

In Table-4.1, we present the results obtained from multiple algorithms (A2C, Q-learning, and Meta-RL) explored in this research. However, our detailed discussion focuses on the outcomes and behavior of A2C.

4.1 Reinforcement Learning Results

The teacher model demonstrated consistent performance improvement during its 20-epoch training process in the knowledge distillation framework. Initially, it achieved a training accuracy of 88.72% with a total loss of 1.2470 and a remarkably high

Table 4.1: Performance comparison of the Knowledge Distillation framework across three stages-(i) baseline with all features, (ii) after feature removal to simulate unlearning, and (iii) after feature restoration, utilizing A2C, Q-Learning, and Meta-RL

Algorithm	Data(%)	Model	Stage	Accuracy(%)	F1 Score(%)	Loss
A2C	25	Teacher	Before Removal	99.599	99.799	0.056
			After Unlearn	99.310	99.654	0.018
			After Restore	99.558	99.778	0.011
		Student	Before Removal	99.602	99.800	0.025
			After Unlearn	99.354	99.676	0.013
			After Restore	99.603	99.801	0.009
	30	Teacher	Before Removal	99.471	99.735	0.146
			After Unlearn	99.388	99.693	0.052
			After Restore	99.471	99.735	0.038
		Student	Before Removal	93.314	96.540	0.048
			After Unlearn	93.163	96.459	0.132
			After Restore	93.314	96.540	0.095
Q-Learning	25	Teacher	Before Removal	99.593	99.796	0.033
			After Unlearn	99.240	99.618	0.071
			After Restore	99.480	99.739	0.057
		Student	Before Removal	99.240	99.618	0.071
			After Unlearn	98.109	99.045	0.136
			After Restore	89.797	99.395	0.114
	30	Teacher	Before Removal	99.667	99.833	0.034
			After Unlearn	99.456	99.727	0.046
			After Restore	99.656	99.828	0.034
		Student	Before Removal	99.456	99.727	0.046
			After Unlearn	99.181	99.588	0.052
			After Restore	99.295	99.646	0.040
Meta-RL	25	Teacher	Before Removal	99.687	99.843	0.020
			After Unlearn	99.445	99.722	0.012
			After Restore	99.687	99.843	0.008
		Student	Before Removal	99.019	99.507	0.048
			After Unlearn	98.912	99.453	0.069
			After Restore	99.019	99.507	0.044
	30	Teacher	Before Removal	99.704	99.852	0.024
			After Unlearn	85.336	92.084	0.333
			After Restore	99.704	99.852	0.007
		Student	Before Removal	98.931	99.463	0.067
			After Unlearn	79.471	88.556	0.125
			After Restore	98.931	99.463	0.039

validation accuracy of 99.60%, indicating strong generalization from the start. The model stabilized around 98.41% training accuracy as training progressed, with a final loss of 0.2115 by epoch 20. Notably, in epoch 4, the validation loss dropped to 0.1869, and by epoch 10, it reached a low of 0.1410, showing stable learning with minimal overfitting. The final evaluation of the test set revealed an impressive accuracy of 99.59%, an F1 score of 99.79%, and a minimal loss of 0.0534, confirming the model’s robustness and high precision-recall balance. In Fig-4.1, we visualized the training accuracy and loss over epochs to illustrate the model’s convergence. The confusion matrix in Fig-4.2 provides deeper insight into the model’s classification performance across classes.

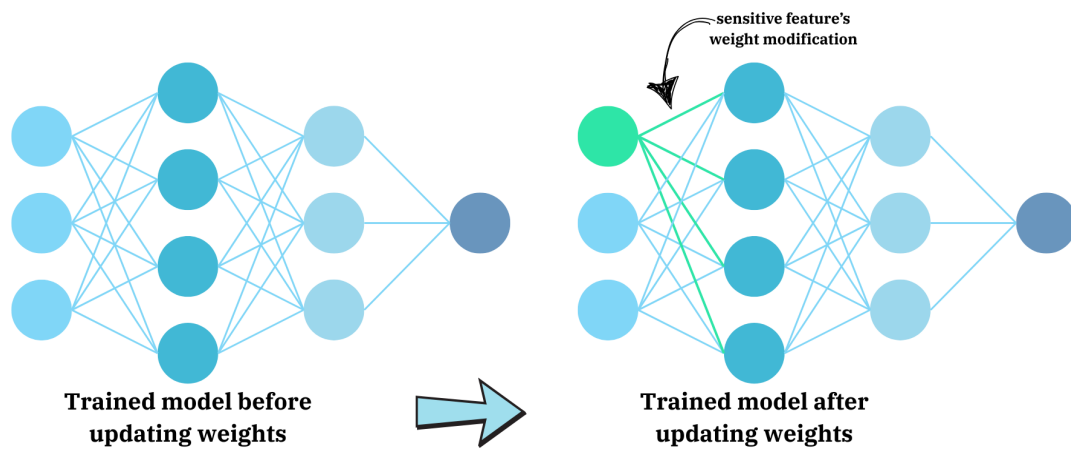


Figure 4.1: Training accuracy and loss at each epoch of Teacher model

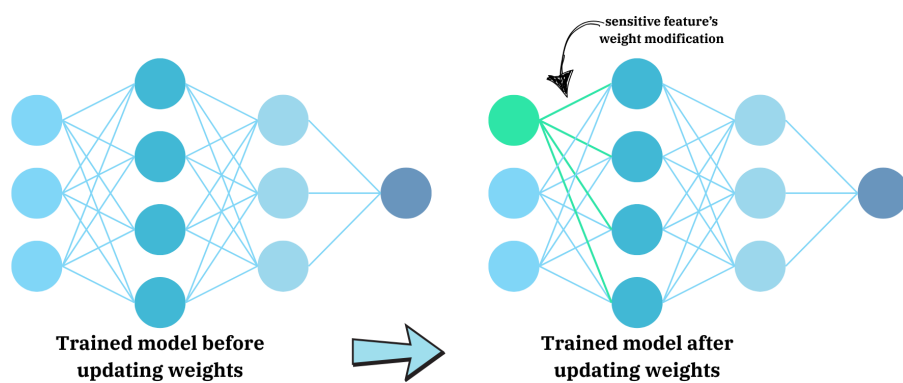


Figure 4.2: Confusion matrix of Teacher model

The student model was trained over 17 epochs and exhibited consistent performance improvements throughout the training process. In the first epoch, it attained a training accuracy of 74.50% with a total loss of 1.0584, while the validation accuracy was already remarkably high at 99.61%, reflecting strong initial generalization. As training progressed, the model’s accuracy steadily improved, converging to a stable level of 97.31% with a final loss of 0.4271 by epoch 17. The validation metrics also demonstrated reliable convergence, with the validation loss reaching its lowest value of 0.3350 in the final epoch, confirming minimal overfitting. Figure-4.3 illustrates the convergence trends of training and validation accuracy and loss across epochs. In the final evaluation, the student model achieved an impressive accuracy of 99.55%, an F1 score of 99.78%, and a minimal test loss of 0.0444. These results highlight the robustness and effectiveness of the knowledge distillation framework, where the student model successfully approximates the teacher model’s performance while maintaining a strong precision–recall balance. For further performance insights, the confusion matrix of the student model is provided in Fig-4.4, offering a detailed view of class-level classification outcomes.

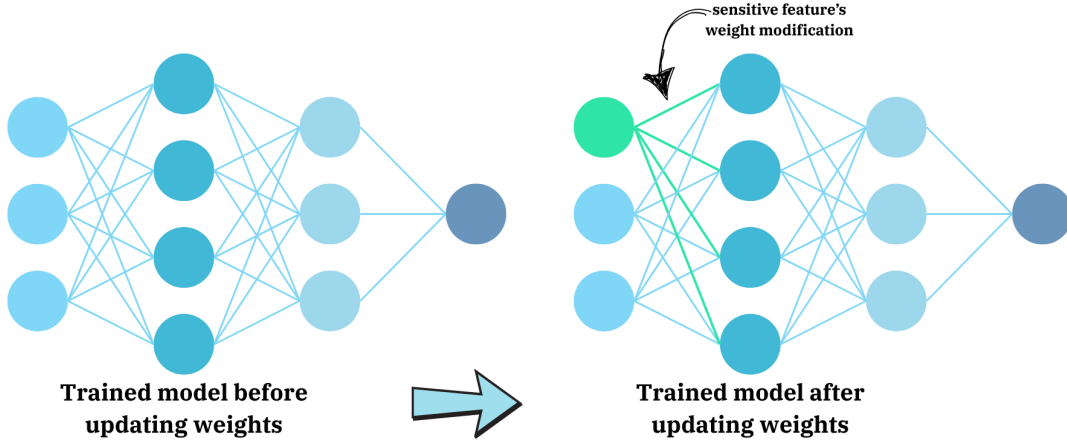


Figure 4.3: Training accuracy and loss at each epoch of Student model

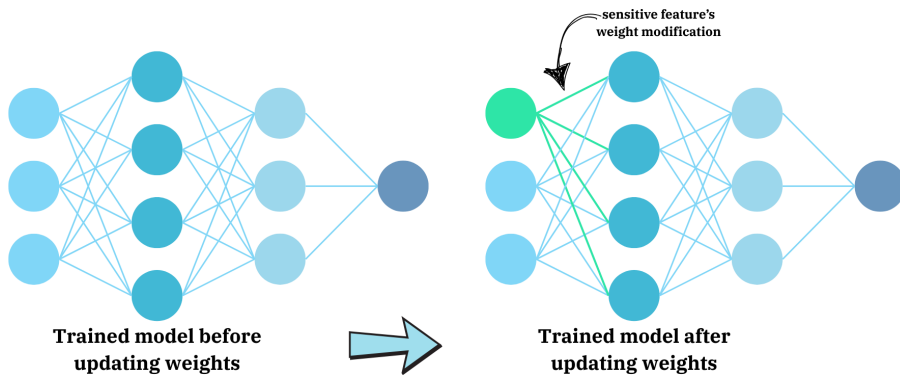


Figure 4.4: Confusion matrix of Student model

4.2 Feature Unlearning Results

In this study, an unlearning technique was used to maintain data security. We first applied the unlearning process to the teacher and student models, which we named

teacher unlearning and student unlearning, respectively.

To strengthen data security, we applied post-hoc weight modification to the first layer of the teacher model. This adjustment ensures the model no longer relies on the removed feature, *flgs* when making predictions. To evaluate the impact of this unlearning process, we re-assessed the model’s performance using accuracy and F1-score. The results showed that the unlearned teacher model achieved an accuracy of 99.35% and an F1 score of 99.67%, with a minimal loss of 0.0156. Unlike previous unlearning attempts that involved highly influential features, the limited performance drop, in this case, suggests that *flgs* had a relatively smaller impact on overall predictions. Nonetheless, this outcome confirms that the model effectively eliminated reliance on the targeted feature. This method improves both model interpretability and data security while maintaining high performance. We have visualized the comparison between the original and unlearned teacher models in Fig-4.5. Additionally, to provide visibility of class-wise prediction behaviour, we generated the confusion matrix of the unlearned teacher model, as shown in Fig-4.6.

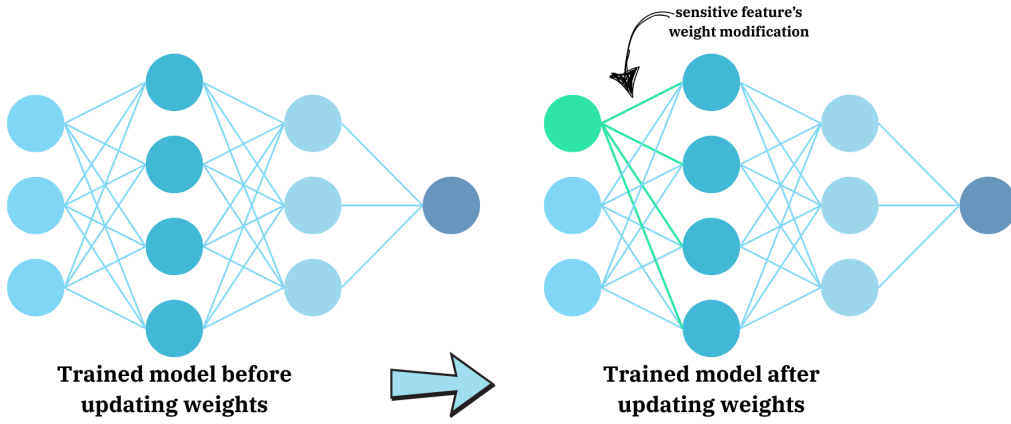


Figure 4.5: Comparison of Teacher and Teacher unlearn models: accuracy, loss and f1 score

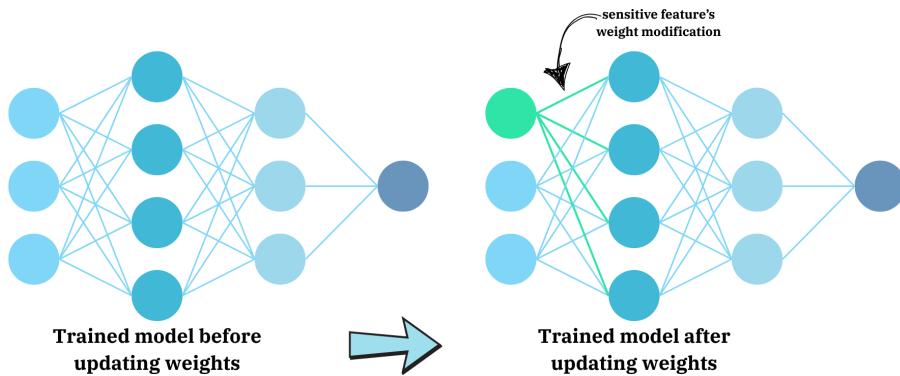


Figure 4.6: Confusion matrix of Teacher unlearn model

To improve data security in the student model, we applied post-hoc weight modification to the first layer of the network. To evaluate the impact of this adjustment, we re-assessed the model using accuracy and F1-score. The results showed that the unlearned student model achieved a test accuracy of 99.35% and an F1 score of 99.67%, with a total loss of 0.0213. The minimal decrease in both metrics indicates that the

model successfully unlearned the influence of the removed feature *flgs*, which had a relatively limited effect on its decision-making. This unlearning approach maintains high performance while improving interpretability and data privacy. We have visualized the performance comparison between the original and unlearned student models in figure-4.7. Additionally, to highlight the class-wise behaviour of the unlearned model, we generated its confusion matrix as shown in fig-4.8.

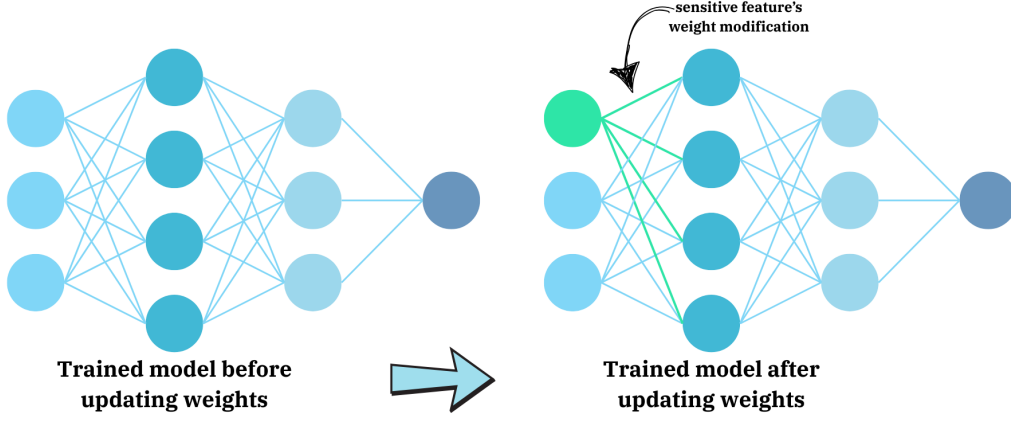


Figure 4.7: Comparison of Student and Student unlearn models: accuracy, loss and f1 score

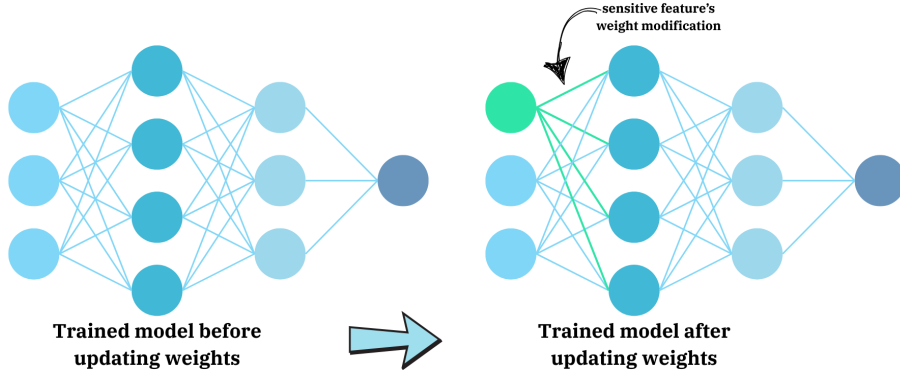


Figure 4.8: Confusion matrix of Student unlearn model

Nevertheless, to check the robustness of the overall framework, we restored the feature flags we had previously removed and reevaluated both the teacher and student models. After restoring the original weights and biases, the teacher model reached an accuracy of 99.59% and an F1 score of 99.79%, with a very low loss of 0.0099. The student model also performed similarly well, with an accuracy of 99.55%, an F1 score of 99.77%, and a loss of 0.0128. These results show that the unlearning process is fully reversible and does not cause any permanent drop in performance. To clarify this, we visualized the confusion matrices for all three stages: before removing the feature, after removing the feature, and after restoring the feature; all together in one frame, as shown in fig-4.9.

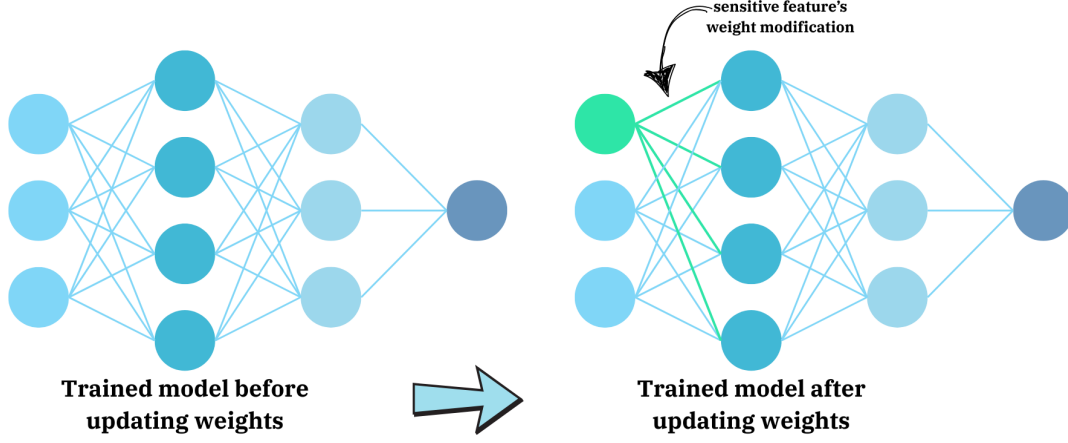


Figure 4.9: Confusion matrices for all three stages: before removing the feature, after removing the feature, and after restoring the feature

4.3 XAI: Teacher, Student and Post-Unlearning

We have applied Explainable AI (XAI) using LIME (Local Interpretable Model-agnostic Explanations) for the model's transparency and visualized which features impacted the most in making such a prediction. We have selected test instance 100 and this instance is an attack traffic.

Using the Teacher model, we have visualized the test case in figure-4.10. In the figure-4.10 illustrates that the teacher model identified test case 100 as an attack with a 100% probability. The feature value highlights feature contributions, with the *state* and *flags* having the highest impact. Other features like *daddr*, *seq*, *sport*, etc., contribute a little, while most others have minimal effects. Additionally, in fig-4.11, we have visualized the top 12 feature impacts.

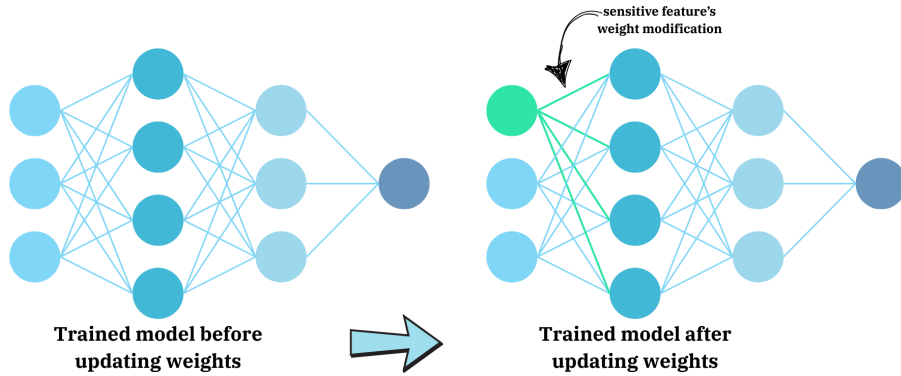


Figure 4.10: XAI interpretation of the Teacher model's prediction for an attack case

We applied Lime on the Student model to show both test scenarios in figure-4.12. Figure-4.12 shows that the model identified test case 100 as an attack with 97% probability, with the *state* and *flags* having the highest impact. Other features, such as *daddr*, *stddev*, *seq* etc., make a minor contribution. In fig-4.13, we have visualized the top 12 feature impacts of the Student model.

Figure-4.14 represents the prediction of test case-100 of the teacher unlearned model using LIME for interpretability. The model predicts a attack with a probability of

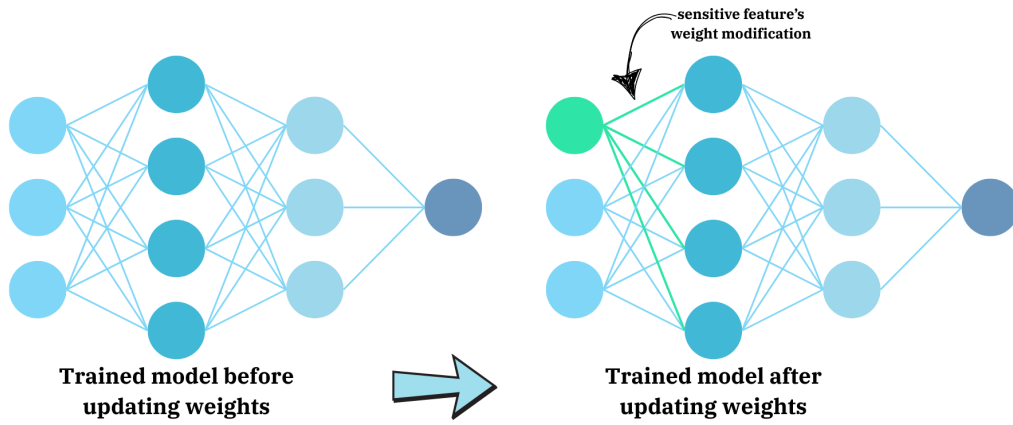


Figure 4.11: XAI interpretation of top 12 features impacts for the Teacher model

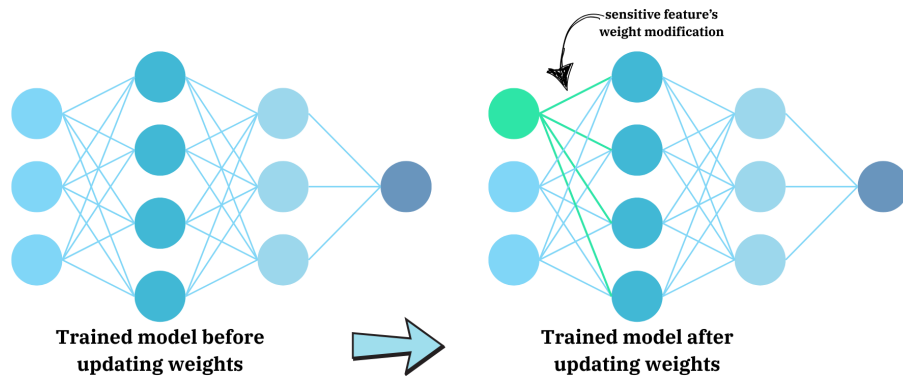


Figure 4.12: XAI interpretation of the Student model's prediction for an attack case

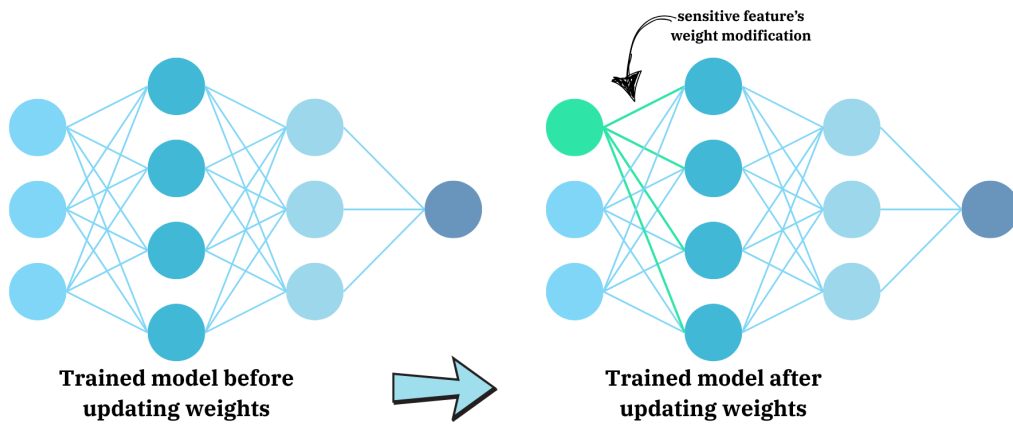


Figure 4.13: XAI interpretation of top 12 features impacts for the Student model

100%. Feature contributions indicate that, *state* and *daddr* influenced the decision the most (as second most dominated feature remove by unlearning technique). However, their impact was not strong enough to alter the classification. Most other features contributed minimally, confirming that the unlearning process did not compromise the model’s decision-making. In fig-4.15, we have visualized the top 12 feature impacts of the Teacher Unlearn model.

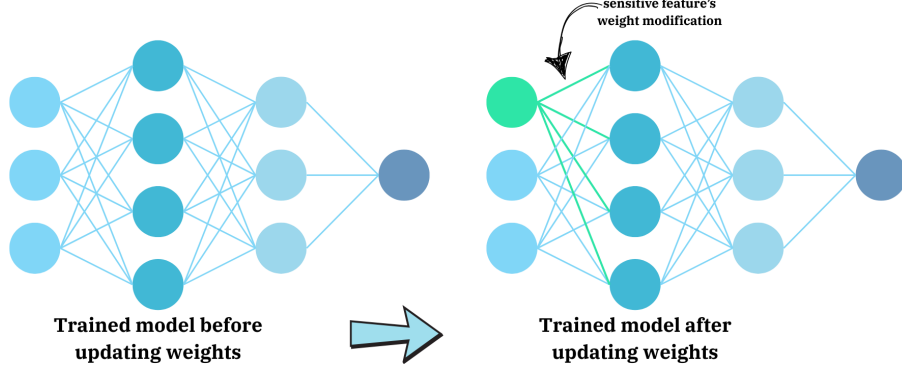


Figure 4.14: XAI interpretation of the Teacher Unlearn model’s prediction for an attack case

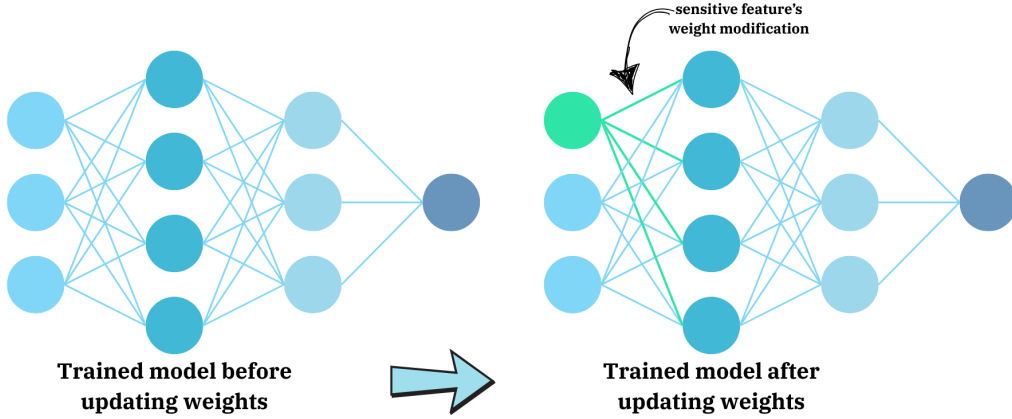


Figure 4.15: XAI interpretation of top 12 features impacts for the Teacher Unlearn

Figure-4.16 represents the prediction of test case-100 of the student unlearning model using LIME for interpretability. The model predicts a attack with a probability of 97%. Feature contributions indicate that, *state* and *pkSeqID* influenced the decision the most (as second most dominated feature remove by unlearning technique). Other features contributed minimally, confirming that the unlearning process did not compromise the model’s decision-making. In fig-4.17, we have illiteratd the top 12 feature impacts of the Student Unlearn model.

4.4 Analysis

This paper presented a compact reinforcement-learning-based botnet detection model, integrating Explainable AI (XAI) and feature unlearning for enhanced security. A key strength is its application of knowledge distillation, enabling the deployment

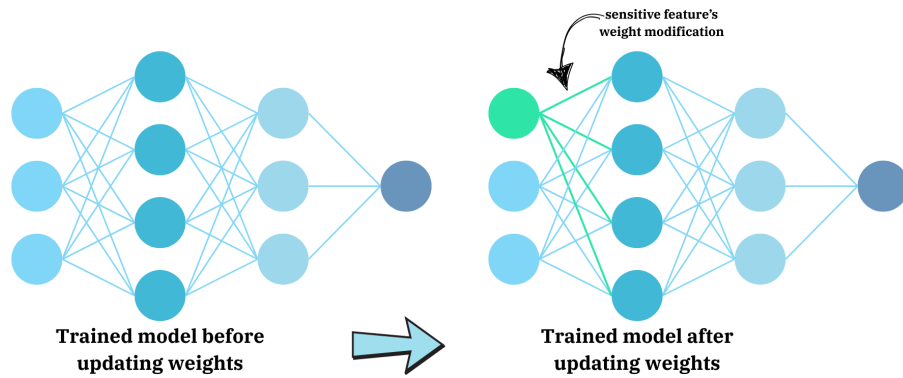


Figure 4.16: XAI interpretation of the Student Unlearn model's prediction for an attack case

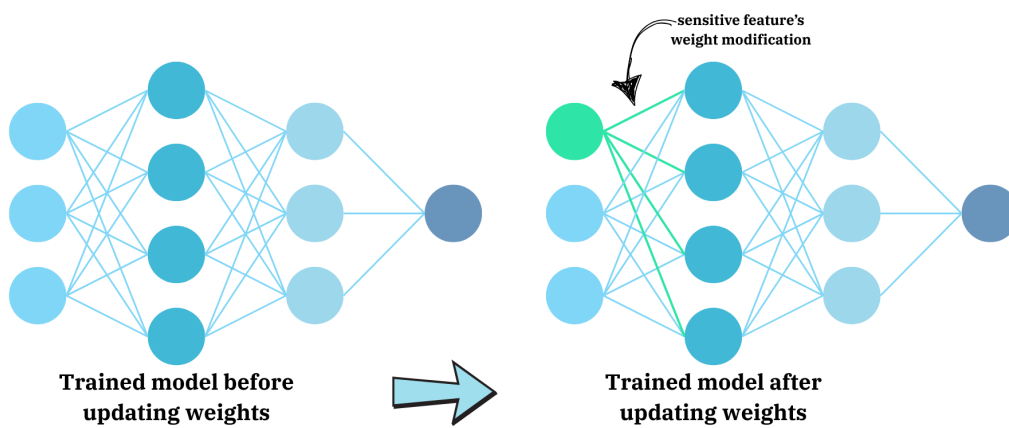


Figure 4.17: XAI interpretation of top 12 features impacts for the Student Unlearn

of an efficient student model while maintaining high accuracy (99.60%) comparable to its teacher model (99.59%). This lightweight approach ensures adaptability in resource-constrained environments, such as IoT devices, surpassing traditional ML-based intrusion detection systems that often require extensive computational resources. Furthermore, integrating XAI techniques like LIME enhances interpretability, addressing deep learning model’s *black-box* nature, a crucial factor for cybersecurity applications [22]. The integration of feature unlearning secure data privacy, a critical aspect in modern AI systems, aligns with recent advancements in machine unlearning [28].

Compared to benchmark models such as ensemble classifiers (e.g., LightGBM) and deep learning approaches like LSTM and RNN, this model exhibits competitive performance while emphasizing data privacy through feature unlearning. Unlike studies that rely on conventional feature engineering or lack a robust privacy-preserving mechanism, this research achieves an effective trade-off between detection accuracy and security compliance. Additionally, its reliance on the BoT-IoT dataset ensures a realistic assessment of cyberattack scenarios, further validating its practical applicability.

The study has limitations that aim for further research. The model primarily focuses on the BoT-IoT dataset, which, while comprehensive, may not capture arising attack variations. Future work could extend its applicability to diverse datasets, ensuring more comprehensive generalizability. Refining advance attention based reinforcement unlearning methods to remove sensitive data dynamically in real time could improve adaptability in evolving threat landscapes. Enhancing the model’s resilience against adversarial attacks through robust feature selection and hybrid learning strategies remains another critical avenue for exploration [2].

4.5 Discussion

To further check the robustness of our framework, we tested the overall framework with 30% of the data instead of just the 25% used earlier. As shown in table-4.1, the teacher and student models performed very well even with the extra data (added 5 million data points). The accuracy, F1 score, and loss values stayed consistently strong at each stage, which includes before feature removal, after feature removal, and after restoring the feature. This shows that the models are accurate and scalable even with big data. For example, with 30% of the data before feature removal, the teacher model reached 99.471% accuracy and a 99.735% F1 score, slightly higher than 25% of the data. Even after removing or restoring the feature, the models gave highly accurate results. These findings confirm that the framework remains effective and scalable.

In this study, we faced many challenges, among which highly imbalanced data distribution was one of the significant. The imbalanced data distribution made it especially challenging to maintain high recall and precision for the benign class, which contained far fewer samples than the attack classes. Nevertheless, our framework achieved consistently strong performance across all evaluation metrics, demonstrating both robustness and adaptability. Feature analysis revealed that the state feature was particularly dominant, exerting greater influence than other features and posing

a risk of bias if not properly managed. One of the framework’s main strengths lies in its lightweight and scalable design, enabling efficient training and deployment even in resource-constrained environments. In addition, the ability to apply post-hoc feature unlearning without retraining gives it a distinct advantage over conventional models. Unlike prior work, Cer-FeaUn [32] has demonstrated certified feature unlearning in federated learning, while RFUL [34] has addressed user-level unlearning and recovery in federated learning. In contrast, this paper applies unlearning to IoT botnet detection through distilled reinforcement learning with explainability. Together, these capabilities allowed our approach to outperform several existing network security models in terms of accuracy, flexibility, and privacy-awareness. Overall, the results confirm the practicality and effectiveness of the framework for real-world, dynamic IoT security scenarios.

Chapter 5

Conclusion

In this study, we developed DiRLU, a lightweight and scalable reinforcement-learning framework designed to improve IoT security. The framework combines knowledge distillation, which enables a smaller student model to inherit the knowledge of a larger teacher model, with a feature unlearning mechanism that protects sensitive data without requiring retraining. DiRLU can learn efficiently while maintaining privacy, making it ideal for small, resource-constrained IoT devices. Unlike benchmark models limited to 5% of the BoT-IoT dataset, we utilized a comprehensive 25% subset, achieving 99.60% accuracy and a 99.80% F1 score. These results prove that DiRLU maintains high performance even when working with larger data samples. Our findings show that feature unlearning is both effective and reversible: removing sensitive features reduced the model's dependence on them while preserving reliability, and restoring these features recovered the original performance. This flexibility ensures that the system can adapt to changing privacy needs without a significant impact on accuracy. The addition of explainable AI further clarified the decision-making process, increasing trust and transparency in model predictions. This helps users and organizations understand why certain decisions are made, which is crucial for security applications. Overall, DiRLU stands out as a practical and adaptable solution for IoT security. Its lightweight design, privacy-preserving capability, and interpretability make it suitable for real-world deployment. Further work will focus on improving its responsiveness to new types of attacks and strengthening defenses against adversarial techniques to support secure and trustworthy IoT solutions.

Bibliography

- [1] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “Smote: Synthetic minority over-sampling technique,” *Journal of artificial intelligence research*, vol. 16, pp. 321–357, 2002.
- [2] M. Farrokhmanesh and A. Hamzeh, “A novel method for malware detection using audio signal processing techniques,” in *2016 Artificial Intelligence and Robotics (IRANOPEN)*, IEEE, 2016, pp. 85–91.
- [3] M. T. Ribeiro, S. Singh, and C. Guestrin, ““why should i trust you?” explaining the predictions of any classifier,” in *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*, 2016, pp. 1135–1144.
- [4] L. H. Gilpin, D. Bau, B. Z. Yuan, A. Bajwa, M. Specter, and L. Kagal, “Explaining explanations: An overview of interpretability of machine learning,” in *2018 IEEE 5th International Conference on data science and advanced analytics (DSAA)*, IEEE, 2018, pp. 80–89.
- [5] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, “Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-iot dataset,” *Future Generation Computer Systems*, vol. 100, pp. 779–796, 2019.
- [6] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, “Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-iot dataset,” *Future Generation Computer Systems*, vol. 100, pp. 779–796, 2019.
- [7] A. Uprety and D. B. Rawat, “Reinforcement learning for iot security: A comprehensive survey,” *IEEE Internet of Things Journal*, vol. 8, no. 11, pp. 8693–8706, 2020.
- [8] J. Brandt and E. Lanzén, “A comparative review of smote and adasyn in imbalanced data classification,” 2021.
- [9] J. Gou, B. Yu, S. J. Maybank, and D. Tao, “Knowledge distillation: A survey,” *International Journal of Computer Vision*, vol. 129, no. 6, pp. 1789–1819, 2021.
- [10] J. L. Leevy, J. Hancock, T. M. Khoshgoftaar, and J. Peterson, “Detecting information theft attacks in the bot-iot dataset,” in *2021 20th IEEE International Conference on Machine Learning and Applications (ICMLA)*, IEEE, 2021, pp. 807–812.

- [11] E. Muhati and D. B. Rawat, “Asynchronous advantage actor-critic (a3c) learning for cognitive network security,” in *2021 Third IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*, 2021, pp. 106–113. DOI: 10.1109/TPSISA52974.2021.00012
- [12] A. Uprety and D. B. Rawat, “Reinforcement learning for iot security: A comprehensive survey,” *IEEE Internet of Things Journal*, vol. 8, no. 11, pp. 8693–8706, 2021. DOI: 10.1109/JIOT.2020.3040957
- [13] M. M. Alani and A. I. Awad, “Paired: An explainable lightweight android malware detection system,” *IEEE Access*, vol. 10, pp. 73 214–73 228, 2022.
- [14] X. Chen, Z. Zhang, and L. Liu, “Improved knowledge distillation with teacher-student adaptation for efficient neural network compression,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 9, pp. 4240–4251, 2022. DOI: 10.1109/TNNLS.2022.3156492
- [15] B. Deepa and K. Ramesh, “Epileptic seizure detection using deep learning through min max scaler normalization,” *Int. J. Health Sci*, vol. 6, pp. 10 981–10 996, 2022.
- [16] A. Kumar, M. Shridhar, S. Swaminathan, and T. J. Lim, “Machine learning-based early detection of iot botnets using network-edge traffic,” *Computers & Security*, vol. 117, p. 102 693, 2022.
- [17] S. Alosaimi and S. M. Almutairi, “An intrusion detection system using bot-iot,” *Applied Sciences*, vol. 13, no. 9, p. 5427, 2023.
- [18] CoverLink, *Mirai ddos attack on dyn case study*, Accessed: 2024-11-06, 2023. [Online]. Available: <https://coverlink.com/case-study/mirai-ddos-attack-on-dyn/>
- [19] IBM, *Reinforcement learning*, Accessed: 2024-11-06, 2023. [Online]. Available: <https://www.ibm.com/think/topics/reinforcement-learning>
- [20] TensorFlow, *Actor-critic methods — tensorflow reinforcement learning*, Accessed: 2024-11-06, 2023. [Online]. Available: https://www.tensorflow.org/tutorials/reinforcement_learning/actor-critic
- [21] Y. Zhang, Z. Lu, F. Zhang, H. Wang, and S. Li, “Machine unlearning by reversing the continual learning,” *Applied Sciences*, vol. 13, no. 16, p. 9341, 2023.
- [22] A. Bensaoud, J. Kalita, and M. Bensaoud, “A survey of malware detection using deep learning,” *Machine Learning With Applications*, vol. 16, p. 100 546, 2024.
- [23] A. Galli, V. La Gatta, V. Moscato, M. Postiglione, and G. Sperli, “Explainability in ai-based behavioral malware detection systems,” *Computers & Security*, vol. 141, p. 103 842, 2024.
- [24] GDPR.eu, *What is gdpr?* Last checked on 19.07.2025, 2024. [Online]. Available: <https://gdpr.eu/what-is-gdpr/>
- [25] C. Miao, L. Kou, J. Zhang, and G. Dong, “A lightweight malware detection model based on knowledge distillation.,” *Mathematics (2227-7390)*, vol. 12, no. 24, 2024.

- [26] P. A. Networks, *What is a botnet?* Last checked on 12.06.2024, 2024. [Online]. Available: <https://www.paloaltonetworks.com/cyberpedia/what-is-botnet>
- [27] C. Patsakis, F. Casino, and N. Lykousas, “Assessing llms in malicious code deobfuscation of real-world malware campaigns,” *Expert Systems with Applications*, vol. 256, p. 124912, 2024.
- [28] W. Wang, Z. Tian, C. Zhang, and S. Yu, “Machine unlearning: A comprehensive survey,” *arXiv preprint arXiv:2405.07406*, 2024.
- [29] Y. Cui and M. H. Cheung, “The price of forgetting: Incentive mechanism design for machine unlearning,” *IEEE Transactions on Mobile Computing*, pp. 1–14, 2025. DOI: 10.1109/TMC.2025.3582904
- [30] S. Hakemi, N. Akhtar, G. M. Hassan, and A. Mian, “Post-hoc spurious correlation neutralization with single-weight fictitious class unlearning,” *arXiv preprint arXiv:2501.14182*, 2025.
- [31] S. S. Johari, N. Shahriar, M. Tornatore, R. Boutaba, and A. Saleh, “Anomaly detection and localization in nfv systems by utilizing masked-autoencoder and xai,” *IEEE Transactions on Mobile Computing*, pp. 1–18, 2025. DOI: 10.1109/TMC.2025.3582195
- [32] Y. Wang, Z. Lu, Z. Liu, T. Li, Z. Chen, and W. Susilo, “Cer-feau: Certified feature unlearning in vertical federated learning,” *IEEE Transactions on Mobile Computing*, pp. 1–13, 2025. DOI: 10.1109/TMC.2025.3594851
- [33] G. Wu and X. Wang, “A privacy-enhanced framework with deep learning for botnet detection,” *Cybersecurity*, vol. 8, no. 1, p. 9, 2025.
- [34] C. Zhou, C. Pan, M. Li, and P. Wang, “Federated unlearning with fast recovery,” *IEEE Transactions on Mobile Computing*, vol. 24, no. 10, pp. 9709–9725, 2025. DOI: 10.1109/TMC.2025.3563265